



TILERA BOARD TEST KIT GUIDE FOR TILE-GX PROCESSORS

DOC. No. UG422
DOC. RELEASE 1.5
September 2014
TILERA CORPORATION

Copyright © 2011-2014 Tileria ® Corporation. All rights reserved. Printed in the United States of America.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, except as may be expressly permitted by the applicable copyright statutes or in writing by the Publisher.

The following are registered trademarks of Tileria Corporation: Tileria and the Tileria logo.

The following are trademarks of Tileria Corporation: Embedding Multicore, The Multicore Company, Tile Processor, TILE Architecture, TILE64, TILEPro, TILEPro36, TILEPro64, TILEExpress, TILEExpress-64, TILEExpressPro-64, TILEExpress-20G, iMesh, TileDirect, TILExtreme-Gx, TILExtreme-Gx Duo, TILEmpower, TILEmpower-Gx, TILEmpower-Gx36, TILEmpower-Gx72, TILEncore, TILEncorePro, TILEncore-Gx, TILEncore-Gx9, TILEncore-Gx16, TILEncore-Gx36, TILEncore-Gx72, TILE-Gx, TILE-Gx9, TILE-Gx16, TILE-Gx36, TILE-Gx72, TILE-Gx8072, TILE-Gx3000, TILE-Gx5000, TILE-Gx8000, TILE-Gx8009, TILE-Gx8016, TILE-Gx8036, TILE-Gx3036, DDC (Dynamic Distributed Cache), Multicore Development Environment, Gentle Slope Programming, TMC (Tileria Multicore Components), hardwall, Zero Overhead Linux (ZOL), MiCA (Multicore iMesh Coprocessing Accelerator), and mPIPE (multicore Programmable Intelligent Packet Engine). All other trademarks and/or registered trademarks are the property of their respective owners.

Third-party software: The Tileria IDE makes use of the BeanShell scripting library. Source code for the BeanShell library can be found at the BeanShell website (<http://www.beanshell.org/developer.html>).

This document contains advance information on Tileria products that are in development, sampling or initial production phases. This information and specifications contained herein are subject to change without notice at the discretion of Tileria Corporation.

No license, express or implied by estoppels or otherwise, to any intellectual property is granted by this document. Tileria disclaims any express or implied warranty relating to the sale and/or use of Tileria products, including liability or warranties relating to fitness for a particular purpose, merchantability or infringement of any patent, copyright or other intellectual property right.

Products described in this document are NOT intended for use in medical, life support, or other hazardous uses where malfunction could result in death or bodily injury.

THE INFORMATION CONTAINED IN THIS DOCUMENT IS PROVIDED ON AN "AS IS" BASIS. Tileria assumes no liability for damages arising directly or indirectly from any use of the information contained in this document.

Publishing Information:

Document Number:	UG422
Document Release Number:	1.5
Date	15 September 2014

Contact Information:

Tileria Corporation
Information info@tilera.com
Web Site http://www.tilera.com

Contents

PREFACE

About this Manual	vii
Intended Audience	vii
New Features	vii
Manual Contents Description	vii
Related Documents	viii
Application Notes	viii
Release and Patch Notes	viii
Data Sheets	viii
User Guides	viii
Technical or Customer Support	ix
Product Information	ix
Notation Conventions	ix

CHAPTER 1 OVERVIEW

1.1 About the Board Test Kit	1
1.2 The BTK's Purpose	1
1.3 BTK Users	1
1.4 Locating the Kit	1

CHAPTER 2 SYSTEM REQUIREMENTS AND BASIC INSTALLATION INSTRUCTIONS

2.1 Requirements	3
2.2 Installation Instructions	3
2.2.1 Installing the MDE	3
2.2.2 Installing Drivers	3
2.2.3 Installing Ipmitool	3
2.3 Installing the BTK	4

CHAPTER 3 STEPS FOR BRINGING UP YOUR NEW BOARD

3.1 Create custom Directory	6
3.2 Add Entry for New Board to custom.py	6
3.3 Create <new_board>.py	6
3.4 Verify Power, Clocks, and Reset	6

3.5 Start the BTK	6
3.6 Verify Host-to-Board Communication	7
3.7 Test the Tile-Gx Processor	7
3.8 Program the BIB	7
3.9 Bring Up DDR3 Memory	8
3.10 Finish Testing Other Board Components	8

CHAPTER 4 BTK STARTUP OPTIONS

4.1 Bringing Up the User Interface	9
4.2 BTK Options	9
4.2.1 Selecting Chip Access with --dev	9
4.2.1.1 Configuration Files	10
4.2.2 Selecting TILE-Gx Node Access via BMC proxy with --dev	11
4.2.3 Selecting Serial UART Chip Access with --serial	11
4.2.4 Selecting TILE-Gx Serial UART Device with --serial-device	11
4.2.5 Selecting BMC Console with --serial-bmc	11
4.2.6 Selecting the BMC Interface with --manage	11
4.2.7 Selecting Board Type with --board	12
4.2.8 Selecting Chip Type with --chip	12
4.2.9 Selecting log Directory with --log	13
4.2.10 Rarely Used Options	13
4.3 Using Customized Python Code	14
4.3.1 Custom Options	14

CHAPTER 5 BTK COMMANDS

5.1 General Commands	15
5.2 Commands Common to all Units	16
5.3 Commands for TILES	18
5.3.1 Tile SPRs	18
5.3.2 Tile General Purpose Registers	19
5.3.3 Other Tile Members	20
5.4 PCIe Commands	20
5.4.1 TRIO Commands	21
5.4.2 TRIO MAC Commands	21
5.4.3 TRIO RC Commands	22
5.5 mPIPE Commands	23
5.5.1 MAC Commands	24
5.5.1.1 Commands for MACs in GbE Mode	24
5.5.1.2 Commands for MACs in XAUI Mode	24
5.6 Rshim I ² C Master Commands	25
5.6.1 I ² C Access – I2CM vs. I2cAccess Classes	25
5.6.2 Access via I ² C Master Interfaces	25
5.6.3 Access via I2cAccess	27
5.7 Displaying Information about I ² C Interfaces	28

5.8 Board Devices	29
5.9 GPIO BTK Commands	29

CHAPTER 6 RUNNING BTK TESTS

6.1 Overview	31
6.2 Test Categories	31
6.2.1 Basic	31
6.2.2 Phase1	31
6.2.3 Phase2	31
6.2.4 Linux	31
6.3 Regression Scripts	31
6.3.1 Regression Tests	32
6.4 Board Devices and Self Tests	33
6.4.1 Self-Tests	33
6.5 Logging	33
6.5.1 What Is Logged	33
6.5.2 Setting Logging and Print Levels	33
6.5.3 Test Logs	34
6.5.4 Log Organization	34
6.5.4.1 Suppressing Output to the Display	35

CHAPTER 7 SETTING UP YOUR BOARD AND CONFIGURING YOUR BIB

7.1 Introduction	37
7.2 Configuring the BTK to Work with a Custom Board	38
7.3 Registering a Board Class	39
7.4 Adding New SROMs	40
7.5 Adding a New PHY	40
7.6 Adding Temperature Sensor Classes	41
7.7 Advanced Custom Configurations	42
7.7.1 Multiple or Undefined Chip Variants	42
7.7.2 Multiple Custom Board Classes	42

CHAPTER 8 WORKING WITH THE DDR3 INTERFACE

8.1 Setting Up the Board Information Block	43
8.2 Configuring a Memory Interface and Running a Test	43
8.3 Using Common Commands to Specify Memory Configuration Options	44

APPENDIX A: COMMANDS MATRIX TABLE

APPENDIX B: BOOTING THE MDE OVER UART

B.1 Introduction	51
B.2 Preparation	51
B.3 Booting	54
B.4 Booting Examples	55

B.5 Using the BTK after Boot57

APPENDIX C: BTK TEST LIST

C.1 Additional BTK Tests59

 C.1.1 Basic Tests 59

 C.1.2 Phase 1, Phase 2, and Linux Tests 62

APPENDIX D: TEST QUALIFIERS

D.1 Selecting Tests73

D.2 Running Qualified Tests73

D.3 Selecting Test Phases74

D.4 Current List of Test Qualifiers74

APPENDIX E: GLOSSARY, CONVENTIONS AND STANDARDS

E.1 Glossary77

INDEX 79

PREFACE

About this Manual

This guide documents usage of the TILE-Gx™ Board Test Kit (BTK) to test TILE-Gx processors, and Tilera® TILE-Gx platforms, to aid in the custom TILE-Gx board bring-up procedures, and to extend the GXBTK with new tests and functionality.

Intended Audience

This manual is intended for use by hardware engineers, software engineers, and field application engineers as they board bring up a board, while they debug software, and when they monitor board operations, for example for power monitoring.

New Features

This version of the BTK supports the following new features:

- Addition of [“Selecting BMC Console with --serial-bmc” on page 11](#) and [“Selecting the BMC Interface with --manage” on page 11](#)
- Addition of [“Selecting Chip Type with --chip” on page 12](#), [“Selecting log Directory with --log” on page 13](#), [“Rarely Used Options” on page 13](#), and [“Custom Options” on page 14](#)
- Support for all TILEncore-Gx™ products, including the TILEncore-Gx9™, TILEncore-Gx16™, and TILEncore-Gx72™ made possible via changes to the Auto-discovery feature, as described in [“Configuring the BTK to Work with a Custom Board” on page 38](#)
- Modifications to [Adding Temperature Sensor Classes](#) on page 41.
- Instructions for Creating I2cAccess Classes, as described in [“I2C Access – I2CM vs. I2cAccess Classes” on page 25](#) and related procedures for using newly-created classes, as described in [“Access via I2cAccess” on page 27](#)
- New example code in [“Registering a Board Class” on page 39](#)

Manual Contents Description

This manual is organized as follows:

- The [Preface](#) (*this chapter*) provides an overview of this manual and information about contacting customer support.
- [Chapter 1: Overview](#) describes the Board Test Kit, its intended audience, and information about where BTK files can be accessed.
- [Chapter 2: System Requirements and Basic Installation Instructions](#) defines hardware and software requirements for using the BTK.
- [Chapter 3: Steps for Bringing up your New Board](#) provides step-by-step instructions for working with the BTK.

- [Chapter 4: BTK Startup Options](#) provides simple instructions for launching the BTK.
- [Chapter 5: BTK Commands](#) lists the supported BTK commands and describes their function.
- [Chapter 6: Running BTK Tests](#) describes each of the test included in the BTK.
- [Chapter 7: Setting up your Board and Configuring your BIB](#) focuses on how to create a Board Information Block (BIB) for a board you have created.
- [Chapter 8: Working with the DDR3 Interface](#) describes how to configure DDR3 memory interfaces.
- [Appendix A: Commands Matrix Table](#) lists new/renamed/deprecated BTK commands.
- [Appendix B: Booting the MDE over UART](#) explains how to boot the MDE on a TILE-Gx processor using a UART (serial) port.
- [Appendix C: BTK Test List](#) provides a comprehensive list of BTK tests as well as descriptions for each.
- [Appendix D: Test Qualifiers](#) provides a preliminary description of qualifiers.
- [Appendix E: Glossary, Conventions and Standards](#) provides terminology introduced in this book.
- [Index](#)

Note: Several examples are provided with the kit. It is suggested that you study these examples to understand how to use the kit. The examples demonstrate the principles and expand on the information provided in this guide.

Related Documents

Additional documentation from Tiler Corporation is available including the following:

Application Notes

- *Application Note: Tiler Board Information Block Format (AN040)*
- *Application Note: Bringing Up a TILE-Gx DDR3 Interface Using the BTK (AN045)*
- *Application Note: Improving DDR3 Timing Margins through RDLAT Training (AN057)*

Release and Patch Notes

- *Multicore Development Environment Release Notes (UG1108)*
- *Multicore Development Environment Patch Notes (UG1118)*

Data Sheets

- *TILE-Gx36 Tile Processor™ Preliminary Data Sheet (DS400)*
- *TILE-Gx16 Tile Processor™ Preliminary Data Sheet (DS401)*
- *TILE-Gx9 Tile Processor™ Preliminary Data Sheet (DS403)*
- *TILE-Gx72 Tile Processor™ Preliminary Data Sheet (DS405)*

User Guides

- *Getting Started with the MDE (UG504)*
- *MDE User Guide (UG501)*

- *TILEncore-Gx Card User's Guide (UG416)*
- *TILEmpower-Gx36 Platform User's Guide (UG415)*
- *TILExtreme-Gx Platform Quick Start and Installation Guide (UG409)*
- *TILExtreme-Gx36 Duo Platform Quick Start and Installation Guide (UG411)*
- *TILEmpower-Gx36 FR Platform User's Guide (UG419)* (not currently available)

Technical or Customer Support

You can reach Tiler Corporation Customer Support in the following ways:

- Visit the Tiler Web site at
<http://www.tilera.com/>
- E-mail questions to
support@tilera.com

Product Information

You can obtain product information from the Tiler Corporation Web site, from the product CD-ROM, or from the printed publications (manuals).

Tiler Corporation is online at <http://www.tilera.com/>.

Notation Conventions

Text conventions used in this manual are as follows:

Example	Description
Close command (File menu)	Titles in reference sections indicate the location of an item within the IDE's menu system (for example, the Close command appears on the File menu).
Write In Progress (WIP) bit	Courier text indicates the names of: • A bit or bitfield, for example the CHANNEL bit • A special purpose register (SPR), for example the RSH_COORD SPR • Code sample • Application, for example the tile-monitor application • Command, for example the link command • State, for example the IDLE state
<u>RSHIM MMIO registers</u>	Blue and underlined text in Courier font indicates these are hypertext links to HTML files associated with the interface, or links to a specific web site.
Chapter 7: XAUI MAC Interface	Blue text in text font (Palatino) indicates a (cross-reference) Hypertext link to text elsewhere in the manual. If you are reading a soft-copy of this manual, you can click on the link to jump directly to the referenced section.
{this that}	Alternative required items in syntax descriptions appear within curly brackets and separated by vertical bars; read the example as this or that. One or the other is required.
[this that]	Optional items in syntax descriptions appear within brackets and separated by vertical bars; read the example as an optional this or that.
[this,...]	Optional item lists in syntax descriptions appear within brackets delimited by commas and terminated with an ellipse; read the example as an optional comma-separated list of this.
.SECTION	Commands, directives, keywords, and feature names are in text with Arial font.

Example	Description
<filename>	Non-keyword placeholders appear in text in italic style format and within brackets.
	Note: For correct operation, ... A Note provides supplementary information on a related topic. In the online version of this book, the word Note appears instead of this symbol.
	Caution: Incorrect device operation might result if ... Caution: Device damage might result if ... A Caution identifies conditions or inappropriate usage of the product that could lead to undesirable results or product damage. In the online version of this book, the word Caution appears instead of this symbol.

CHAPTER 1 OVERVIEW

This Board Test Kit (BTK) guide is intended for developers to verify board functionality and develop tests for boards based on a Tile Processor™ (a member of the TILE-Gx™ processor family).

1.1 About the Board Test Kit

The BTK is a collection of Python modules, C code, and assembly code to provide a means for low level testing and debug of Tile-Gx based systems. It includes many tests for the Tile-Gx processor and its I/O ports, as well as tools to aid in debug of boards or applications.

The BTK is not a full board diagnostic suite—it provides a baseline for customer tests. Customers can modify the BTK for their own needs.

1.2 The BTK's Purpose

The BTK provides:

- A baseline board test kit for Tilera® designed boards and Tile Processors.
- The elements to support custom boards designed by Tilera customers or partners.
- Methods to extend or add tests to enhance board screening and support new hardware.
- The simplest, fastest tool to verify Tile Processor and board functionality.
- The best tool to debug, identify, and isolate issues.

1.3 BTK Users

In addition to Tilera's engineers who use the BTK to test boards designed by Tilera, it can be used by:

- Engineers evaluating Tile Processors, Tilera boards like the TILEncore-Gx™ or platforms like the TILEmpower-Gx™, or reference designs.
- Developers using or modifying BTK for board production tests.
- Engineers debugging hardware issues.

1.4 Locating the Kit

The BTK is provided as part of the MDE and the BTK files are part of the directory structure.

CHAPTER 2 SYSTEM REQUIREMENTS AND BASIC INSTALLATION INSTRUCTIONS

2.1 Requirements

The board test kit runs on a Linux host with the following requirements:

- Tilera Multicore Development Environment™ (MDE) installed

Note: The IDE is *not* required in order to use the BTK.

- Python version 2.4.3 or higher
- Tilera PCIe driver installed (for boards with PCIe functionality)
- Tiler USB driver installed (for boards connected via USB)
- Connection from host to board under test (either USB, PCIe, or Serial UART). Note that USB or PCIe are preferred for best performance.
- The board under test installed in a PCIe slot of host (for boards with PCIe functionality)
- BMC Ethernet and BMC UART Console (if the BMC is present).¹

The Board Test Kit (BTK) relies on USB, UART, or PCIe to access Tile Processor diagnostic and control registers, and to execute diagnostic programs on the processor.

Note: If the board under test is connected by UART to the host, the UART protocol mode must be enabled using the UART Configuration Interface pins, `CONFIG_UART[2:0]`.

2.2 Installation Instructions

2.2.1 Installing the MDE

For instructions on installing the MDE, refer to “How to Install the MDE Software” in the *MDE Getting Started Guide* (UG504).

2.2.2 Installing Drivers

For instructions on installing drivers, refer to “Installing the x86 Drivers” in the *MDE Getting Started Guide* (UG504).

2.2.3 Installing Ipmitool

The `ipmitool` is available and maintained at <http://ipmitool.sourceforge.net/>.

1. Note that the `ipmitool` is installed for systems with a BMC present (these requirements are optional and are used to run tests that interact with the BMC).

To install under CentOS use:

```
yum install OpenIPMI OpenIPMI-tools  
chkconfig ipmi on  
service ipmi start
```

2.3 Installing the BTK

The kit is provided as part of the Tiler MDE, and is invoked with the following script:

```
${TILER_ROOT}/bin/tile-btk
```

The source Python files, C and assembly source for tests, support files, etc. can be found in the following directory:

```
${TILER_ROOT}/lib/tile-btk/
```

CHAPTER 3 STEPS FOR BRINGING UP YOUR NEW BOARD

This chapter describes the steps you should take when first bringing up your new Tile-Gx™ based board.

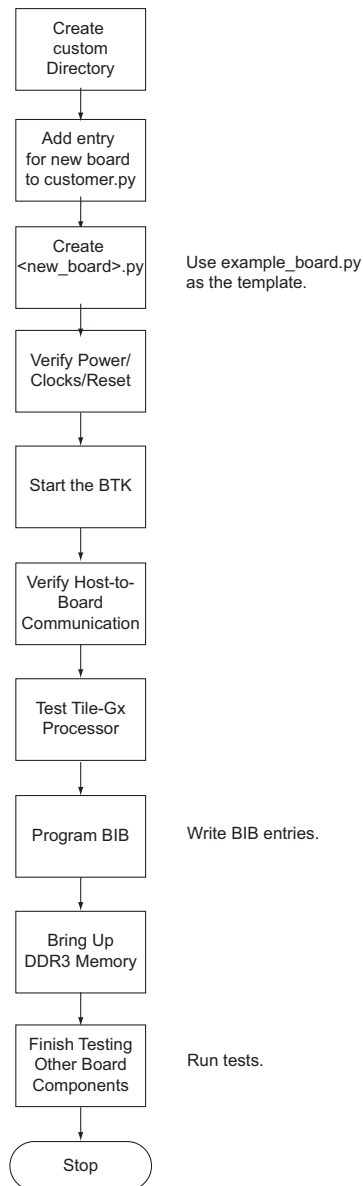


Figure 3-1. New Board Bringup Workflow

3.1 Create custom Directory

Copy `$TILERA_ROOT/lib/tile-btk/example_custom` to your custom directory.

3.2 Add Entry for New Board to custom.py

In the file that contains custom board class(es), import the Python files. This file contains a template `parts2class` method that can be used to Auto-discover multiple board types. The default board class that BTK uses (even if auto-discovery fails to find a matching board), is specified in this file. Typically before importing it, the only change that needs to be made in this board file involves modifying this line.

If board auto-discovery capabilities are desired for multiple custom board classes, it can prove easier to use multiple custom directories. If you want to use the `parts2class` method to discover multiple board classes (as is done with Tiler platforms), register the board classes and list of part numbers in the custom board file.

For more information, see [“Configuring the BTK to Work with a Custom Board” on page 38](#).

3.3 Create <new_board>.py

Create `<new_board>.py` using `example_board.py` as the template. For more information, see [“Configuring the BTK to Work with a Custom Board” on page 38](#).

3.4 Verify Power, Clocks, and Reset

Before even starting the BTK, power up your new board and verify with a scope that the power supplies, clocks and resets are functioning properly.

3.5 Start the BTK

This step assumes you have updated the btk with your own board-related customizations described in [Chapter 7, Setting up your Board and Configuring your BIB](#).

1. Connect your new board to the host where the Tiler® Multicore Development Environment (MDE) is installed, using PCIe (if you have an add-in card), USB, USB via an IP proxy service, or serial UART.
 - If you are using PCIe or USB, make sure the appropriate driver provided by Tiler is installed on the host. `--dev` specifies only USB, PCIe, or USB via IP chip access.
 - If you are using UART, the `--serial` flag must be used to indicate UART is the primary chip access.
 - If an interface configuration file (see [“Configuration Files” on page 10](#)) is available, the serial device name is specified in this file; otherwise, `--serial-device <device name>` must also be specified.

2. Run the BTK by typing on the host command line:

```
$TILERA_ROOT/bin/tile-btk [--dev <device>]
```

If only UART is available, run BTK by typing:

```
$TILERA_ROOT/bin/tile-btk [--serial --serial-device <device>]
```

The *device* option is not required if your board is the only board connected to the host and there is only one connection type from the host to the board. If you have multiple board classes in your custom directory or if auto-discovery is not working, specify your board class and chip type on the host command line to bypass hardware discovery or board match dependencies:

```
$TILERA_ROOT/bin/tile-btk --board <your_board> --chip <your_chip> [--dev <device>]
```

Refer to [Chapter 5, BTK Commands](#) for more details on `tile-btk` command line options.

3.6 Verify Host-to-Board Communication

The BTK relies on the connection from the host to the board under test to communicate with the Tile-Gx's RShim interface, and from there to communicate with the rest of the system. Ensure that connection is valid before trying anything else by typing at the `btk` prompt:

```
btk> rshim.test()
```

If that fails, ensure the Tile-Gx processor is receiving good power and a valid clock, and check the connection between your board and the host.

3.7 Test the Tile-Gx Processor

Next test that the Tile-Gx processor has basic functionality by typing the following at the `btk` command line:

```
btk> chip.alive()
```

This test verifies that all the major blocks of the processor are reachable, that the chip can be reset, and that the clock inputs are the correct frequency.

If that passes, you are ready to run the simplest tests that boot the tiles.

You can run tests individually. For example:

```
btk> tests["l2_data"].run()
```

See [Chapter 6, Running BTK Tests](#) for more details on running individual tests.

Or you can instruct the `btk` to run all “simple” tests (meaning tests that do not require any resources outside of the boot path and the Tile-Gx processor itself. Most notably, these tests do not require that DDR3 RAM is working.)

```
btk> board.gx_regress()
```

3.8 Program the BIB

Before you can go further in testing your board, you need to program the Board Information Block (BIB).

Please refer to [Chapter 7, Setting up your Board and Configuring your BIB](#) for instructions on creating the correct BIB for your board and programming it into your onboard ROM.

3.9 Bring Up DDR3 Memory

You are now ready to test the DDR3 memory on your board. [Chapter 8, Working with the DDR3 Interface](#) describes how to do that.

For more information, refer to *Application Note: Bringing Up a TILE-Gx DDR3 Interface Using the BTK* (AN045). Add any necessary BIB entries to define DDR3 functionality based on the results of the Read latency (RDLAT) search.

3.10 Finish Testing Other Board Components

Now that DDR3 is working, you can run tests that require DDR3.

Again, you can run them individually. For example:

```
btk> tests["shared_mem"].run()
```

Or you can run all tests you excluded in the step above that require DDR3.

```
btk> board.set_qual(qualifiers="memory", disqualifiers=("development", "manual", "custom", "ext"))
btk> board.run_tests()
```

The btk also has a built in function that runs all provided TILE-Gx tests:

```
btk> board.gx_regress()
```

You can also run all provided board tests (includes memory, SRAM, and I²C ROMs) as follows:

```
btk> board.board_regress()
```

For more information, refer to [Section 3.7 “Test the Tile-Gx Processor”](#) on page 7.

CHAPTER 4 BTK STARTUP OPTIONS

4.1 Bringing Up the User Interface

The user interface of the test kit should be invoked as follows:

```
$TILERA_ROOT/bin/tile-btk
```

When the user interface starts, the Python command-line interpreter will start and present the following prompt:

```
btk>
```

At this point, you can invoke any of the supported commands. See *Chapter 5, BTK Commands*.

Scripts of Python instructions can be executed from the command line using the Python built-in `execfile()` function. For example:

```
btk> execfile("my_script.py")
```

Other options can be specified on the command line. Type `$TILERA_ROOT/bin/tile-btk -h` for a full list.

For more information about using customized modules, refer to [“Using Customized Python Code” on page 14](#).

4.2 BTK Options

4.2.1 Selecting Chip Access with `--dev`

The `--dev {NAME|LABEL}` option allows a user to specify a chip access interface directly by `NAME` or by an alias `LABEL`.

Configuration files are optional, but they are useful for identifying chip access interfaces on hosts with multiple Tiler processors.

If `tile-btk` is started without the `--dev` option, BTK will attempt to start with the USB or PCIe device or configuration file that it finds. The `--dev {HOST#NODE}` option also allows a user to specify a chip access made available through a management interface (usually a BMC) by specifying the host name or IP address and TILE-Gx node.

The `--dev` option is not used for specifying UART chip access. Instead, the `--serial` option indicates to BTK to use the UART device specified in an optional configuration file or by the `--serial-device` parameter.

Options

NAME	This option includes devices like usb0 or gxpci1 (added by the Tiler USB or Tiler PCIe driver).
LABEL	This option implies that a configuration file such as /etc/tileusb-empower0.conf or /etc/tilepci0.conf exists that provides a label to identify the interface.
none	This indicates that no chip access is to be used when starting BTK. Note that most BTK tests and functions rely on chip access. Notes: • Used alone (--dev none), BTK is run in a utility mode with limited capabilities. • Used in tandem with the --board and the --chip options, this can verify instantiation of the Board Class, without accessing the chip. • Used with both --board and --manage options; BMC interactions are available.

Examples

<code>tile-btk</code>	Starts BTK on a host with only one usb/pcie device or configuration file.
<code>tile-btk --dev none</code>	Starts BTK without chip access.
<code>tile-btk --dev empower3</code>	Starts BTK with a USB or PCIe instance referenced in the configuration file for empower3.
<code>tile-btk --dev gxpci0</code>	Starts BTK with PCIe instance 0.
<code>tile-btk --dev hostname#1a</code>	Starts BTK on CPU module 1a.

4.2.1.1 Configuration Files

A configuration file might look like this:

```
serial=/dev/ttyS0
label=empower1
manage=mybmc
```

As an alternative, you can also use the following file. This is similar to `serial` (above), but instead of defining the TILE-Gx Console, it defines the BMC Console.

```
label=tileusb7
serial=/dev/ttyUSB2
bmc_serial=/dev/ttyUSB0
manage=10.4.0.241
```

For boards with multiple chip access choices (such as USB and PCIe), a configuration file can exist for each interface with the secondary interface type as part of the label in the configuration file.

Default interface configuration file of `/etc/tilepci_empower0.conf`:

```
serial=/dev/ttyS0
label=express0
```

Secondary interface configuration file of `/etc/tileusb_empower0.conf`:

```
serial=/dev/ttyS0
label=express0-usb
```

4.2.2 Selecting TILE-Gx Node Access via BMC proxy with --dev

This section describes how to use the BTK to access TILEExtreme-Gx nodes via the BMC. The `--dev {HOSTNAME and CPU}` option allows a user to specify a node access interface directly by HOSTNAME and TILE-Gx node as follows:

```
--dev <hostname-or-ip-address>#<node>
```

For example: `--dev 173.11.255.5#1a`

Note that the hostname or IP address is the BMC's hostname or IP address.

You can determine the IP address of the BMC at boot time. The BMC briefly displays its DHCP IP on the console. For more information, refer to the *TILEExtreme-Gx Platform Quick Start and Installation Guide* (UG409).

4.2.3 Selecting Serial UART Chip Access with --serial

When you use `--serial`, it dictates that the UART device indicated in the configuration file with `serial=` or specified with `--serial-device` be used instead of the chip access indicated by a configuration file or the `--dev` parameter.

4.2.4 Selecting TILE-Gx Serial UART Device with --serial-device

Use `--serial-device <device_name>` to specify the Serial UART device file to be used.

Note: Typically this is specified in the event that UART is necessary for chip access (using the `--serial` option), but it can also be used for interaction with the TILE-Gx Console.

Examples

```
tile-btk --serial-device /dev/ttyS0
```

Note: If UART is the only access available to the TILE-Gx Processor on your board, you will also need to boot the MDE using UART. See [Appendix B: Booting the MDE over UART](#) for a detailed instructions.

4.2.5 Selecting BMC Console with --serial-bmc

Use `--serial-bmc <bmc_console_name>` to specify the Serial UART device you want to use to interact with the BMC Console.

4.2.6 Selecting the BMC Interface with --manage

You can specify the BMC or management interface, which is typically an IP address or alias. This interface can be used by some tests to query for temperature or other board status information under BMC supervision/control or to power cycle a board.

4.2.7 Selecting Board Type with --board

The `--board <board-type>` option has two important functions:

- First, it prevents any hardware accesses until explicitly requested by the user at the `bt>` prompt in combination with the `--chip` parameter. Auto-discovery of the chip and board devices does not occur.
- Second, the board attributes are determined by the board type selected, but chip attributes are still Auto-discovered, unless the `--chip` option is also used.

Note that `--board help` displays all board types that can be specified with this option. Many of these appear in a part number format that is typically found on an Hardware Identification (HID) label affixed to the unit.

If you are using the `--custom` option to load custom board classes into BTK, those classes are also displayed. See [“Using Customized Python Code” on page 14](#).

Using this option instead of board auto-discovery is preferred if you are:

- Debugging an issue and do not want to change chip or board state by starting BTK (must also use the `--chip` option)
- Running production tests
- Programming a BIB for a custom board

Examples

```
tile-btk --board empower1
tile-btk --dev empower5 --board TILEmpower-Gx36
tile-btk --dev empower5 --board TILEmpower-Gx36 --chip gx8036-12ce
```

4.2.8 Selecting Chip Type with --chip

The `--chip <chip-type>` option has two important functions:

- First, it prevents any hardware accesses until explicitly requested by the user at the `bt>` prompt when used in combination with the `--board` parameter. Auto-discovery of the chip and board devices does not occur.
- Second, chip attributes are selected by the chip type specified, but board attributes are Auto-discovered unless the `--board` option is also used.

`--chip help` displays all chip types that can be specified with this option. The first column displays a short-hand version of the chip types. The second column displays the actual name of the chip, which can also be used to select a given chip type.

Using this option instead of chip auto-discovery is preferred if:

- You are debugging an issue and do not want to change chip or board state by starting BTK (must also use the `--board` option).
- You want to verify that a chip is operating exactly as expected, and you do not want to trust auto-discovered parameters.

You see something like the following dump of the possible chips:

```
-bash-3.2$ ./btk --dev usb7 --chip help
Valid CHIP TYPES:
gx8072-12ce      TLR4-07280DG-12CE
gx5036-10c       TLR4-03650CG-10C
gx8072-10ce      TLR4-07280DG-10CE
gx8016-10c       TLR4-01680CG-10C
gx5009-10c       TLR4-00950CG-10C
gx8036-10c       TLR4-03680CG-10C
gx8016-12ce      TLR4-01680CG-12CE
gx8036-12c       TLR4-03680CG-12C
gx5009-12c       TLR4-00950CG-12C
gx5036-12c       TLR4-03650CG-12C
gx5016-12ce      TLR4-01650CG-12CE
gx5009-10ce      TLR4-00950CG-10CE
gx8009-10ce      TLR4-00980CG-10CE
gx8009-12ce      TLR4-00980CG-12CE
gx5036-12ce      TLR4-03650CG-12CE
gx8016-10ce      TLR4-01680CG-10CE
gx8016-12c       TLR4-01680CG-12C
gx5036-10ce      TLR4-03650CG-10CE
gx5016-10c       TLR4-01650CG-10C
gx5036-12lce     TLR4-03650CG-12LCE
gx8009-10c       TLR4-00980CG-10C
gx5016-12c       TLR4-01650CG-12C
gx8009-12c       TLR4-00980CG-12C
gx5009-12ce      TLR4-00950CG-12CE
gx5016-10ce      TLR4-01650CG-10CE
gx8072-10c       TLR4-07280DG-10C
gx8072-12c       TLR4-07280DG-12C
gx8036-12ce      TLR4-03680CG-12CE
gx8036-10ce      TLR4-03680CG-10CE
```

4.2.9 Selecting log Directory with --log

If the `--log` option is not used, logs are saved to a temporary directory and deleted upon exiting BTK. See [Section 6.5, "Logging"](#) for more information. To access the local directory, use `--log`. Note that you do not explicitly need to specify the name and location of the log file.

For additional information on how and where logs are maintained and managed, refer to [Section 6.5 "Logging"](#) on page 33.

4.2.10 Rarely Used Options

The following options are infrequently used:

- `--no-completion` is specified if tab completion should be disabled.
- `--bmc {HOSTNAME}` starts up BTK with a system interface for platforms with multiple TILE-Gx nodes (all accessible through a BMC management interface) to run tests that might affect more than one TILE-Gx node during its execution. This is typically used only in a production environment.¹
- `--node <1 - N or sys>` is used to specify the board class for a TILE-Gx node where multiple TILE-Gx nodes are present on a single board accessible by the Host via USB. If `sys` is specified instead of a node number, board classes for all nodes are instantiated in a system context allowing tests that rely on inter-node or system interaction to run.

1. Note that the term *node* as used in this manual is interchangeable with CPU_nA or CPU_nB (where multiple CPUs are present).

4.3 Using Customized Python Code

If you have one or more custom modules (e.g. `custom.py`) that you would like to use within the BTK environment, place them in a directory (e.g. `my_customizations`) and then use the `--custom` option to include that directory in the import search path:

```
btk ... --custom=my_customizations ...
```

You can then use the Python `import` command to load the module and make its contents accessible from the `btk>` prompt:

```
btk> import custom
btk> custom.my_function("Hello!")
Hello!
```

The most common form of customization is defining a board class for your Tile-Gx based system. An example of how to do this is provided in the directory `${TILERA_ROOT}/lib/tile-btk/example_custom`.

If you want to load a Python script directly into the current environment, you can use the Python built-in `execfile()` function instead. For example:

```
btk> execfile("my_script.py")
my_function("Hello!")
Hello!
```

See [Chapter 7, Setting up your Board and Configuring your BIB](#) for details on expanding BTK to include custom board configurations and tests.

4.3.1 Custom Options

The `--custom-options` option allows for board-specific custom options specified for BTK to be used with custom board classes.

CHAPTER 5 BTK COMMANDS

5.1 General Commands

The most commonly used BTK commands are listed in [Table 5-1](#). Using the `dir()` command, all interfaces that can be accessed are listed. Similarly, using `dir(interface)` lists all the objects accessible within a given interface. Using the `help(interface)` command, methods available for each interface are listed as well as details for using those methods (the method's purpose, input variables, and outputs). Usage of some of these interfaces is also present later in this section.

Table 5-1. General BTK Commands

Commands	Description
<code>board.set_core_voltage()</code>	Specifies the core voltage in volts.
<code>board.set_ddr_voltage()</code>	Specifies the DDR3 voltage in volts.
<code>board.get_core_voltage()</code>	Returns the setting of the core voltage in volts.
<code>board.get_ddr_voltage()</code>	Returns the DDR3 voltage in volts.
<code>board.set_clock_voltage_pairs()</code>	Sets core clock rates and corresponding voltages as a function of frequency for core or specified shim.
<code>board.i2c_discovery()</code>	Discovers I ² C devices on i2cm busses.
<code>board.chip_access_points</code>	Displays valid chip access interfaces.
<code>board.switch_chip_access(access)</code>	Changes chip access to another device ('uart', 'gxpci', 'usb', 'bmc').
<code>board.get_cpu_temperature()</code>	Displays TILE-Gx temperature sensors (if under GX control).
<code>chip.set_core_clock_rate(rate)</code>	Sets the chip's core clock rate in Hz.
<code>chip.core_clock_rate()</code>	Returns the core's current clock rate in MHz.
<code>chip.reset()</code>	Issues soft reset to TILE-Gx processor.
<code>chip.alive()</code>	Runs basic aliveness test.
<code>chip.calc_home(pa)</code>	Calculates the home tile for a given physical address assuming hash-for-home memory. Uses the rshim Home-Map table.

Table 5-1. General BTK Commands (continued)

Commands	Description
<code>chip.measure_refclk(measure_seconds=4.0)</code>	Measures the refclk rate in MHz.
<code>chip.print_network_state(network)</code>	Checks network and print picture of network blockages, if any. "network" is "QDN", "RDN", "SDN", "UDN", or "IDN"; if no network is given, all five networks are checked.
<code>chip.print_pcs()</code>	Prints the current PC of all tiles in a grid format.
<code>chip.print_pcs_and_lrs()</code>	Prints the current PC and LR of all tiles in a grid format.
<code>chip.rev()</code>	Returns revision ID of Gx processor.
<code>chip.dump_f2v()</code>	Displays the frequency-to-voltage table.
<code>chip.show()</code>	Prints graphic showing chip I/O and clock rates.
<code>chip.cmem_read(pa, size=8, hfh=False, x=0, y=0, mask=15, offset=0)</code>	Reads memory from home tile. This assumes that the Rshim Address Mapping Table (AMT) has been populated if hfh is True.
<code>chip.perf_rates()</code>	Report the event rates of all cores for some key performance statistics.
<code>board.show_tests()</code>	List all available tests.
<code>board.print_all_test_status()</code>	Displays all cumulative test errors.
<code>board.clear_all_test_status()</code>	Clears cumulative test status.
<code>dir()</code>	Lists all the interfaces that you can use. ^a
<code>help()</code>	Provides information on an interface. Use this command with a parameter, such as <code>board(help(board))</code> . This command identifies the interface type, its function, and all the related commands for that interface, including any supported parameters. When you use the <code>show_test</code> parameter (<code>help(board.show_tests())</code>), it provides more detailed information about one method.

a. An interface is defined as a method of accessing functionality, for example: `board`, `chip`, `usb`, `pcie0`.

5.2 Commands Common to all Units

The BTK provides the ability to read and write registers throughout the TILE-Gx processor.

The following objects (corresponding to chip units) provide a `dump()` routine to read and print out all their registers and other state helpful for debug:

Table 5-2. Top-Level BTK Objects

Objects	Description
<code>comp[n]</code> ^a	Compression/Decompression Engine.
<code>crypto[n]</code>	Encryption/Decryption Engine.
<code>gpio</code>	FlexIO.
<code>mpipe[n]</code>	Ethernet, XAUI, etc.
<code>msh[n]</code>	DDR3 interface.
<code>rshim</code>	Various low speed I/O and chip control.
<code>rsh_i2cm[n]</code>	I ² C Masters.
<code>rsh_i2cs</code>	I ² C Slave.
<code>rsh_uart[n]</code>	RS232 UARTs.
<code>trio[n]</code>	PCIe and StreamIO.
<code>tile(x,y)</code>	Tile

a. Where [n] is the shim number, with the first being “0”.

For example:

```
trio[n].dump()
```

Each of these objects contain an object called `regs`, under which is instantiated an object for every register.

To show a single register’s decoded fields, call `dump()` on an individual register:

```
unit_name.regs.register_name.dump()
```

Where `unit_name` matches those above, and `register_name` is the same as is displayed by `unit_name.dump()`. For example:

```
trio[n].regs.clock_count.dump()
```

For even more detail about a register, call `describe()` on it instead of `dump()`. This will give a description of each field of the register. For example:

```
trio[n].regs.clock_count.describe()
```

There are two ways to write a register with the BTK.

1. To write the entire register with a value:

```
unit_name.regs.register_name.write(<value>)
```

2. To change just one field, it is a 3-step process

```
a. unit_name.regs.register_name.read()
b. unit_name.regs.register_name.field_name = value
c. unit_name.regs.register_name.write()
```

In addition to the dump routine, each unit provides helpful debug routines specific to its function. To see a list of available routines and a brief description, type:

```
help(unit_name)
```

More detail on the most commonly accessed units and routines is provided below.

5.3 Commands for TILES

The BTK provides access to various bits of tile state using the chip's diagnostics interfaces, including Special Purpose Registers (SPRs) and general purpose register values.

The following commands operate on one tile. Note that tiles can be specified using either `tile(x,y)` or `cpu(n)` syntax, where `n` corresponds to the Linux CPU number.

Table 5-3. Commands for TILES

Commands	Description
<code>tile(x,y).dump(verbose=True)</code>	Dumps out the subset of the tile's state, which is most likely to be useful when debugging. (If verbose is True, dump everything.) Note that the tile is automatically quiesced when this is called, and stays quiesced until <code>resume()</code> is called for that tile.
<code>tile(x,y).quiesce()</code>	Quiesces the tile. Note that by default, this stops instruction issue (the SBox) but not coherence actions (the CBox). You can quiesce the CBox as well by adding <code>cbox=True</code> as an argument to this function. For example: <code>tile(x,y).quiesce(self, cbox=False)</code>
<code>tile(x,y).resume()</code>	Un-quiesces the tile.
<code>tile(x,y).read_va(va)</code>	Uses the associated Tile's TLB to determine Physical Address (PA) and homing, then do a coherent read to the home Tile.
<code>tile(x,y).pa_trace()</code>	Provide set of histograms for cacheline activity in the tile. This can be useful to identify hot cache lines when analyzing performance issues.

5.3.1 Tile SPRs

A tile's Special Purpose Registers (SPRs) are made available in much the same way as other registers in the BTK; they can in general be both read and written, although some SPRs are write-only or read-only. All of the methods used with device configuration registers are available on SPRs; `read()`, `write()`, `dump()`, direct bitfield access, and so forth.

All SPRs are members of the “sprs” member of a Tile object, similar to the `regs` member of an I/O shim object; again, the member name is in lower case and does not contain a leading `SPR_`. As an example, we could examine the contents of an SPR as follows:

```

btk> tile(1,0).sprs.tile_coord.dump()
      Register value = 00000000000040000
      FIELD-NAME [MSB:LSB] = VALUE
      =====
      y_coord    [10:7] = 0x0
      x_coord    [21:18] = 0x1

```

Note that a few SPRs whose names conflict with Python reserved words have `_r` appended to their names; `pass_r` is the most obvious example.

5.3.2 Tile General Purpose Registers

A tile’s general-purpose registers (GPRs) are also available, in much the same way as its SPRs. The tile diagnostics interface does not support modifying GPRs, and there are no bitfields defined for them, so the set of available methods is smaller, but otherwise identical.

All GPRs are members of the `regs` member of a Tile object. Registers `r0` through `r52` are provided as members of an array called `r`; `tp`, `sp`, `lr`, and `pc` are given those names. Some examples:

```

btk> tile(1,0).regs.r[0].read()
8192L
btk> tile(1,0).regs.r[0].dump()
r0 = 0x00000000000002000
btk> tile(1,0).regs.sp.dump()
sp = 0xffffffffc0000efc8
btk> tile(1,0).pc.dump()
pc = 0xffffffffc00000070

```

For convenience, the members of the `regs` member are also members of the Tile object itself, so you can also do things like:

```

btk> tile(1,0).pc.dump()
pc = 0xffffffffc00000070

```

The `regs` member has one method of its own:

```
tile(x,y).regs.dump()
```

Dump out the contents of all the GPRs.

For example:

```

btk> tile(0,0).dump()
Tile (0,0) Registers:
r0  0xffffffff0000e30000  r19 0x000000000000000001  r38 0xfffffffff0000ff00
r1  0xffffffffe0000460980  r20 0xffffffffe0000e30990  r39 0xffffffffe0000efe800
r2  0x000000000000001680  r21 0x000000000000000000  r40 0x000000000000007e00
r3  0xffffffffe0000e30980  r22 0xfffffffffc0010ee68  r41 0xffffffffe0000e104bc
r4  0x000000000000002000  r23 0xfffffffffc0010ede0  r42 0xffffffffe0000eff3e8
r5  0xffffffffe0000460940  r24 0xfffffffffc0010ee70  r43 0xffffffffe0000e30000
r6  0xffffffffe0000460990  r25 0xfffffffffc0010ef40  r44 0x00000000000000100
r7  0xffffffffe0000461fff  r26 0xfffffffffc0010ef48  r45 0xfffff000001000000
r8  0xffffffffe0000460988  r27 0xfffffffffc0010ef50  r46 0x00000000000010000
r9  0x000000000000000000  r28 0xfffffffffc0010efc0  r47 0xfffffffffffff000000
r10 0x000000000000000000  r29 0xfffffffffc0010efc8  r48 0xffffffffe0001000000
r11 0x000000000000000000  r30 0xffffffffe0000d6019c  r49 0xffffffffe0000002ab8
r12 0xffffffffe0000e30000  r31 0xffffffffe0000e30000  r50 0xffffffffe00004652a0
r13 0xffffffffe0000460000  r32 0xffffffffe0000460000  r51 0xffffffffe00004652c0
r14 0xffffffffe0000e30988  r33 0x000002000000000000  r52 0xffffffffe0000463330
r15 0xffffffffe0000e30970  r34 0xffffffffe0000462000  tp  0x00000000000000000
r16 0xffffffffe0000e30968  r35 0xffffffffe0000d76618  sp  0xfffffffffc0000efc8
r17 0xffffffffe0000e30960  r36 0xffffffffe0001000000  lr  0xfffffffff70043b9d8
r18 0xffffffffe0000e30958  r37 0x000000000000000000  pc  0xfffffffffc000000070

```

5.3.3 Other Tile Members

There are other members of the Tile that primarily exist as a way to dump out some interesting data structure.

Table 5-4. Commands for Other Tile Members

Commands	Description
<code>tile(x,y).dtlb.dump()</code>	Dumps out the contents of the DTLB.
<code>tile(x,y).itlb.dump()</code>	Dumps out the contents of the ITLB.
<code>tile(x,y).home_map.dump()</code>	Dumps out the contents of the Home Mapping Table.
<code>tile(x,y).l1i.dump()</code>	Dumps out the contents of the L1 Icache.
<code>tile(x,y).l1d.dump()</code>	Dumps out the contents of the L1 Dcache. Note - valid bits are not accessible via diag methods.

5.4 PCIe Commands

The `trio[n]` unit has several subunits that contain other useful registers for debug. First, there are subunits for each MAC:

`trio[n].mac[m]` (where `m` is 0-2 for TILE-Gx36 and TILE-Gx72)¹

The `dump()` routine can be called on each mac:

```
trio[n].mac[m].dump()
```

1. Where `n` is 0 for TILE-Gx36 and 0 or 1 for TILE-Gx72.

In addition, each mac has subunits specific to what PCIe mode the mac is in (Root Complex or Endpoint).

```
trio[n].mac[m].rc.dump()
trio[n].mac[m].ep.dump()
```

A good starting point for debugging a PCIe or StreamIO port are the `port_config` and `port_status` registers:

```
trio[n].mac[m].regs.port_config.dump()
trio[n].mac[m].regs.port_status.dump()
trio[n].mac[m].regs.stream_config.dump()
trio[n].mac[m].regs.stream_status.dump()
```

The following sections list the most common PCIe commands. Default values for optional parameters are shown.

5.4.1 TRIO Commands

Table 5-5. TRIO Commands

Commands	Description
<code>trio[n].dma_ring_info(idx, push=True)</code>	Prints information related to push_dma ring including if/why the ring is stuck.
<code>trio[n].print_dma_desc(ring, idx, push=True)</code>	Prints one DMA descriptor, given ring number and position in ring.
<code>trio[n].pull_dma_dump()</code>	Dumps out enabled DMA rings for pull.
<code>trio[n].push_dma_dump()</code>	Dumps out enabled DMA rings for push.
<code>trio[n].state()</code>	Prints summary of state of all the MACs on this trio shim.

5.4.2 TRIO MAC Commands

The following commands operate on one MAC, or port. `n` is the MAC number, which can be 0-2 for a TILE-Gx36 or TILE-Gx72.

Table 5-6. TRIO MAC Commands

Commands	Description
<code>trio[n].mac[m].pma_status()</code>	Provides the current port state. This information is useful for initial bring-up to validate that clocks are operating correctly and that the port is trained.
<code>trio[n].mac[m].pcie_init(partial_okay=True, type=None, enter_compliance=False, force_det=False, max_width=8)</code>	Trains the associated PCIe port. The optional type parameter can be set to EP or RC. Otherwise it infers the type based on the strapping pins. If the link partner has not been enabled (for example, when <code>perst</code> is still asserted), a link is established when the link partner is enabled.

Table 5-6. TRIO MAC Commands (continued)

Commands	Description
<code>trio[n].mac[m].config_pcie(type='RC', max_width=8, lane_flip=False, gen1_only=False, disable_scrambler=False, force_det=False, enter_compliance=False)</code>	Configures a port for PCIe operations.
<code>trio[n].mac[m].config_streamio(width='X4', rate='R3125')</code>	Configures a port for StreamIO operations.
<code>trio[n].mac[m].disable_port()</code>	Disables one port.

5.4.3 TRIO RC Commands

The following commands operate on a PCIe port that is in root complex mode. Again, *m* is the MAC or port number.

Table 5-7. TRIO RC Commands

Commands	Description
<code>trio[n].mac[m].rc. clear_device_errors()</code>	Clears out any error status registers in the AdvancedErrorReporting structure in each PCIe device in the PCIe tree.
<code>trio[n].mac[m].rc.crawl (update_busses=True, dotty=False, gen_jpg=False)</code>	Enumerates the PCIe subsystem. RC (root-complex) function to enumerate the PCIe subsystem. It will setup all intervening PCIe switches (peer-to-peer bridges) and assign memory resources. To examine the PCIe bus in a system that was booted via Linux/tile-monitor, set <code>update_busses=False</code>; this will do a more passive crawl. The BAR registers are read/written to determine their size, but are left in their original state. For a completely passive crawl, set <code>bar_size_check=False</code>. The crawler also generates a file called <code>pgraph.txt</code>, which can be processed by a program such as <code>graphviz™</code> in order to produce a picture of the PCIe subsystem. When the crawl is complete, it will print the device memory map, per-device information, and an ASCII view of the PCI tree.
<code>trio[n].mac[m].rc. print_pcie_tree()</code>	Prints the PCIe tree, based on a previous bus crawl. Any link errors are printed with the associated device. This function is called automatically on each crawl but can be subsequently called manually to dump the current state of the tree. The resulting ASCII picture shows all logical PCI busses with their associated PCIe port widths and speeds.

Table 5-7. TRIO RC Commands (continued)

Commands	Description
<code>trio[n].mac[m].rc. find_devices(venid=None, devid=None, min_bus=1)</code>	Finds device(s) matching vendor ID and/or device ID.
<code>trio[n].mac[m].rc. cfg_test_devices()</code>	Performs a quick test of all the devices on the bus to make sure there are not link errors.

5.5 mPIPE Commands

The mPIPE unit has subunits for each mac. To access the registers associated with each use:

```
mpipe[n].mac[m].dump()1
```

Or for more detail on one register:

```
mpipe[n].mac[m].regs.reg_name.dump()
```

Table 5-8 lists more mPIPE commands. Default values for optional parameters are also shown.

Table 5-8. mPIPE Commands

Commands	Description
<code>mpipe[n].bsm_validate()</code>	Ensures Buffer Stack Manager setup and buffers are legal.
<code>mpipe[n].channel_mac_number(channel)</code>	Returns the MAC number associated with this channel.
<code>mpipe[n].dump_counters()</code>	Dumps various counters from the mPIPE unit.
<code>mpipe[n].edma_ring_info(idx)</code>	Prints information related to one eDMA ring including if/why the ring is stuck.
<code>mpipe[n].edma_dump()</code>	Dumps out information for all enabled eDMA rings.
<code>mpipe[n].edma_validate()</code>	Checks the state of the EDMA setup.
<code>mpipe[n].print_edma_desc(ring, idx)</code>	Prints one eDMA descriptor, given ring number and index into ring.
<code>mpipe[n].idma_validate()</code>	Checks the state of the IDMA setup.
<code>mpipe[n].port_info()</code>	Prints summary of information for each port.

1. Where n is 0 for TILE-Gx36 and 0 or 1 for TILE-Gx72.

5.5.1 MAC Commands

The following commands operate on a single MAC in any mode, where *m* is the MAC number.

Table 5-9. MAC Commands

Commands	Description
<code>mpipe[n].mac[m].config()</code>	Configures the MAC.
<code>mpipe[n].mac[m].link_status()</code>	Returns status of one link.
<code>mpipe[n].mac[m].link_up()</code>	Verifies the link status.
<code>mpipe[n].mac[m].phy_crawl()</code>	Looks for PHYs in MDIO space.
<code>mpipe[n].mac[m].verify_pkt_counters(exp_pkt_count=1)</code>	Verifies GbE packet counters.

5.5.1.1 Commands for MACs in GbE Mode

The following commands operate on a MAC that is in GbE mode, where *n* is the MAC number.

Table 5-10. Commands for MACs in GbE Mode

Commands	Description
<code>mpipe[n].mac[m].mac_loopback(loopback=1)</code>	Enables or disables MAC Tx to Rx loopback.
<code>mpipe[n].mac[m].phy_rd(phy_addr, phy_reg, phy_dev=0)</code>	Reads an MDIO PHY register using clause 22.
<code>mpipe[n].mac[m].phy_wr(phy_addr, phy_reg, phy_data, phy_dev=0)</code>	Writes an MDIO PHY register using clause 22.

5.5.1.2 Commands for MACs in XAUI Mode

The following commands operate on a MAC that is in XAUI mode, where *n* is the MAC number.

Table 5-11. Commands for MACs in XAUI Mode

Commands	Description
<code>mpipe[n].mac[m].link_test(double_rate=False, internal_loopback=True)</code>	Enables SerDes/PCS loopback, MAC Rx/Tx and checks the PCS status.
<code>mpipe[n].mac[m].phy_rd(phy_addr, phy_reg, phy_dev=1)</code>	Reads an MDIO PHY register using clause 45.
<code>mpipe[n].mac[m].phy_wr(self, phy_addr, phy_reg, phy_data, phy_dev=1)</code>	Writes an MDIO PHY register using clause 45.

5.6 Rshim I²C Master Commands

5.6.1 I²C Access – I2CM vs. I2cAccess Classes

There are two approaches to communicating with I²C master interfaces. The first involves directly accessing these interfaces and the other involves establishing I2cAccess Classes.

Consider the following:

```

btk> help(rsh_i2cm[0].read)
Help on method __read in module i2cm:
__read(self, dev_addr, dev_offset, length, no_addr=False, restart=False,
dev_addr_16bit=False)

```

There are advantages and disadvantages to each approach, as described below.

Direct access guidelines:

- Any mux channel selection or bus arbitration must be done manually before the I²C device can be accessed.
- Parameters such as `dev_addr_16bit`, `no_addr`, and `restart` must be considered and set appropriately.
- Accessing the I²C device might require in-depth knowledge of I²C or SMBus protocols.
- Accessing the I²C device might require knowledge of the I²C bus layout.

Users might prefer to use the I²C access method, but these are optional classes that facilitate easier, faster command line access to commonly used I²C devices for debug or other device usage (such as reading temperatures or monitoring power).

I2cAccess class access guidelines:

Note: The creation of I2cAccess Classes is not required for board bring-up.

- Mux channels on the I²C device's path are automatically selected
- Default parameters for `dev_addr_16bit`, `no_addr`, and `restart` are provided by the class.
- Class methods display expected I²C devices including full device paths. `i2c_discovery` compares discovered devices to the list of devices managed by I2cAccess classes in the board class.

5.6.2 Access via I²C Master Interfaces

There are two methods in BTK for accessing I²C devices, described in [Table 5-12](#). One is a read function and the other is a write function. The simplest and most direct method is to issue commands directly from I²C master interfaces.

Table 5-12. *rsh_i2cm* Commands

Commands	Description
<pre>chip.rsh_i2cm[i].read(dev_addr, dev_offset, length, no_addr=False, restart=False, dev_ad- dr_16bit=False)</pre>	i I2C Interface number (0-2 for TILE-Gx36). Where device classes include: i2cm0, i2cm1, i2cm2. dev_addr 8-bit slave address of the I²C device. dev_offset Register offset (or command) for the I²C device. length Length of the read, this number of bytes is returned. no_addr If True, dev_offset is ignored. This is used with I²C devices which do not take a register address; many I²C switches fall in this category. restart If True, a start-bit is sent at the end of the transaction instead of a stop-bit; this is useful for SMBUS devices, typically on I²C writes. dev_addr_16bit Set to True for devices that take a 16-bit register address.
<pre>chip.rsh_i2cm[i].write(dev_addr, dev_offset, data, no_addr=False, restart=False, dev_ad- dr_16bit=False)</pre>	i I2C interface number (0-2 for TILE- Gx36). Where device classes include: i2cm0, i2cm1, i2cm2. dev_addr 8-bit slave address of the I²C device. dev_offset Register offset (or command) for the I²C device. data A list of bytes to write (for example, [byte1, byte2, ...]). no_addr If True, dev_offset is ignored. This is used with I²C devices, which do not take a register address; many I²C switches fall in this category. restart If True, a start-bit is sent at the end of the transaction instead of a stop-bit; this is useful for SMBUS devices, typically on I2C writes. dev_addr_16bit Set to True for devices that take a 16-bit register address

5.6.3 Access via I2cAccess

Define I2cAccess classes as described in the following steps:

1. Create one `add_i2cmN_bus()` method per i2cm.
2. Use `add_i2cm_buses()` to create an I2cAccess object per i2cm bus.
3. Define the device mappings for this bus from `i2cm_bus_list[0]`.

This example includes a device of type master; all I2c devices classes automatically arbitrate the bus and select a mux channel necessary for device access.

4. Use the `create_board_devices()` command to create objects that group i2c devices together based on functionality such as:
 - `temp` (temperature sensors),
 - `pmon` (power monitoring)

The following example illustrates how I2cAccess is defined:

```
def __init_board(self):
    """Initialize board configurations
    Create board devices"""
    self.__add_i2cm0_bus()
    self.__add_i2cm2_bus()
    self.add_i2cm_buses()
    self.create_board_devices()
    self.__add_tests()

def __add_i2cm0_bus(self):
    """Assign I2cm0 configuration"""
    rom_boot = [0xA8, 'At24c512b_boot', 'device']
    bus_mst = [0xE0, 'Pca9541', 'master']

    i2c_top_devs = [None, 'channel', bus_mst, rom_boot]
    self.i2cm_bus_list[0] = ['branch', i2c_top_devs]

def __add_i2cm1_bus(self):
    """Assign I2cm1 configuration"""
    spd0 = [0xA0, 'SpdRom_0', 'device']
    spd1 = [0xA4, 'SpdRom_1', 'device']
    i2c_top_devs = [None, 'channel', spd0, spd1]
    self.i2cm_bus_list[1] = ['branch', i2c_top_devs]
```

Note that in the expression:

```
[slave_addr, Class_identifier, type]
[0xA8, 'At24c512b_boot', 'device']
```

A device to be added to the list of I2c devices has four values. In this example these values are defined as:

```
slave address = 0xA8,
device class = At24c512b,
device identifier = boot,
device type = device
```

Note the use of an underscore character (highlighted in red), which separates the class name from a unique identifier. Both of these I2c devices are defined in the SpdRom Class.

I2cAccess classes can be grouped by functionality. Common groups of functionality are Temperature Sensors and Power Monitors, as illustrated below:

```
def __create_board_devices(self):
    """ Create all board objects """

    # Power Monitor configuration
    # List values are Device      Name,          rshunt  vsMax vbus_expected
    pmon_config = [('Ina219_0', 'VRD Vcore',      .001,   0.08, 1.05),\
                   ('Ina219_1', 'GX MEM0',        .01,    0.1, 1.5),\
                   ('Ina219_2', '12V Monitor',     .005,   0.05, 12),\
                   ('Ina219_3', 'Gx 1.0V',         .01,    0.1, 1.0),\
                   ('Ina219_4', 'GX 1.8V',         .01,    0.07, 1.8)]

    # Create Power Monitors
    self.pmon = PowerMonitor(pmon_config, self.i2cm_buses[2].device_map, self)
    self.pmon.i2c_access = self.i2cm_buses[2].i2c_access
    self.board_devices['pmon'] = self.pmon
    top.export("pmon", self.pmon)

    # Create Temp Sensor
    self.temp = TempSensor(\
        self.i2cm_buses[2].device_map, self)
    self.board_devices['temp'] = self.temp
    top.export("temp", self.temp)
```

Above, a PowerMonitor class and TempSensor Class are instantiated. They take as an inputs all I2c devices mapped to I2cm2 and are children of the I2cAccess Class. When these classes are initialized, all devices of the matching type are added to the class that groups their functionality. This is explained further in [Section 5.8 “Board Devices”](#) on page 29.

Afterwards, when temp.read_all_remote() is invoked, it reports temperatures from all I2c temperature remote sensors.

5.7 Displaying Information about I²C Interfaces

Within the TILE-Gx BTK the following I²C interfaces were created to organize I²C Devices for Tiler Board Classes: i2cm0, i2cm1, i2cm2 are I2cAccess objects. These interfaces provide short cuts for interacting with I2c devices with defined i2c device classes.

At the btk prompt, enter the following to display information about I²C devices:

```
btk> i2cm0.show_all()
```

An example of the following is displayed:

Device	Addr	Type	Description	Path [[slave_addr, channel],]
At24c512b	0xa8	device	I2C ROM	['0xa8', None]
Lm95235	0x30	device	Temp Sensor	['0x30', None]
Pca9546	0xe0	mux	4-chan sw	['0xe0', None]
GxSlave	0x20	device	GX I2cs	['0xe0', 0], ['0x20', None]
Max6639GX	0x58	device	Temp Sensor	['0xe0', 3], ['0x58', None]

5.8 Board Devices

A dictionary of board devices (`board.board_devices`) exists for shortcuts to board devices (usually, but not always, I²C devices). Often, individual devices (with shared functionality) are grouped under a single device. Examples of these include `pmon` (Power Monitors), `pmgr` (Power Managers), and `temp` (Temperature Sensors). Use of these shortcuts allows all devices with potentially different hardware access mechanisms but similar functionality to be accessible through a single interface.

Often, methods to read and write registers or execute device specific methods are possible using these device interfaces.

5.9 GPIO BTK Commands

Table 5-1. GPIO BTK Commands

Commands	Description
<code>gpio.read_gpio(pin)</code>	Reads GPIO.
<code>gpio.set_gpio(pin)</code>	Sets GPIO (high).
<code>gpio.clear_gpio(pin)</code>	Clears GPIO (low).
<code>gpio.set_mode(pin, mode)</code>	Sets the state of GPIO pins. Mode options are: • <code>dis</code> <code>disable</code> • <code>out</code> <code>output</code> • <code>in</code> <code>input</code> • <code>od</code> <code>open drain</code> Pin refers to the GPIO[0-63] pins. Refer to the GPIO pin definitions in the TILE-Gx data sheet. Example: <code>set_mode(23, 'out')</code>

CHAPTER 6 RUNNING BTK TESTS

6.1 Overview

To show Board Test Kit (BTK) tests, run `board.show_tests()`. Tests shown are members of a test dictionary, executed as `tests['<test_name>'].run()`, where the test's name is provided by `board.show_tests()` in each of the test types: **Basic**, **Phase1**, **Phase2**, **Linux**.

6.2 Test Categories

6.2.1 Basic

These are Python-only tests. For a list of these tests, refer to [“Basic Tests” on page 59](#).

6.2.2 Phase1

These tests require source code loaded and executed on one or more tiles. These use and exercise L1 and L2 caches and are typically written in assembly. For a list of these tests, refer to [“Phase 1 Tests” on page 62](#).

6.2.3 Phase2

These Phase2 tests boot tiles as Phase1 test do, but they also configure memory and run the test from memory. These tests are typically written in C. For a list of these tests, refer to [“Phase 2 Tests” on page 68](#).

6.2.4 Linux

These tests boot Linux on the chip using the `tile-monitor` command, then run their respective tests under Linux. For a list of these tests, refer to [“Linux Tests” on page 72](#).

These tests can also be run without the BTK, but are included in the BTK as a convenience, so that Linux tests can be included in BTK test scripts.

6.3 Regression Scripts

The following is a list of existing test scripts.

- `board.gx_regress()` Run all the core chip tests (no devices beyond chip are used, including memory).
- `board.board_regress()` Run all core chip, memory, and ROM tests (no manual or external loopback tests).

Most tests in these scripts are in the `tests[]` dictionary executed using the `board.run_tests()` method. These tests are determined by:

1. Selecting appropriate test qualifiers and disqualifiers (see [“Test Qualifiers” on page 73](#)).

2. Selecting the Test Categories (See “Test Categories” on page 31) from using the `phases` parameter.

For example, the `gx_regress` script selects only Basic and Phase1 tests (`board.run_tests(phases=[0, 1])`). The `board.run_test()` method otherwise executes all Test Categories by default.

The `gx_regress` script also executes `chip.alive()` to indicate basic chip capabilities before executing other tests.

The `board_regress` script also executes `board_devices` self tests using `board.test()`. This is similar in concept to `chip.alive()` for devices in the `board.board_device` dictionary. All devices in this dictionary execute their self tests, each of which lightly exercises a device to indicate that it is functional.

Once a script is finished, it might be useful to issue the `board.print_all_test_status()` command to display which tests ran, and their pass/fail status. If a test name has an asterisk before the name (for example: `*Temp Sensor`), this indicates that it is a board device self test (executed in the `board.test()` method) not present in the `tests[]` dictionary.

Here is an example of test status after running `board.test()`. These are all self tests.

```
btk> board.print_all_test_status()
```

	NAME	STATUS	RUNS	FAILS	P/F-RATIO	START	STOP	DURATION(s)
1	*At24c512b_mbd	PASS	1	0	100%	08:03:56	08:03:59	0:00:02
2	*Ethernet PHY	PASS	1	0	100%	08:03:59	08:04:05	0:00:06
3	*Temp Sensor	PASS	1	0	100%	08:04:05	08:04:05	0:00:00
4	*Chip	PASS	1	0	100%	08:04:05	08:04:05	0:00:00
5	*At24c512b_boot	PASS	1	0	100%	08:04:05	08:04:08	0:00:02
6	*At24c512b_AIB	PASS	1	0	100%	08:04:08	08:04:11	0:00:02
7	*DS3645	PASS	1	0	100%	08:04:11	08:04:35	0:00:24
8	*PLX8615	PASS	1	0	100%	08:04:35	08:04:35	0:00:00
9	*RtcPcf8563	PASS	1	0	100%	08:04:35	08:04:39	0:00:04
10	*Pca9555_IOM	PASS	1	0	100%	08:04:39	08:04:39	0:00:00

The list of tests below was created by using test qualifiers and running `board.run_tests()`:

```
btk> board.print_all_test_status()
```

	NAME	STATUS	RUNS	FAILS	P/F-RATIO	START	STOP	DURATION(s)
1	crypto_aes	PASS	1	0	100%	13:37:44	13:38:29	0:02:54
2	l2_data_nocrc	PASS	1	0	100%	13:38:30	13:38:36	0:00:07
3	l2_dapaths	PASS	1	0	100%	13:38:37	13:38:43	0:00:07
4	regfile_write	PASS	1	0	100%	13:38:44	13:38:50	0:00:07
5	srom	PASS	1	0	100%	13:38:51	13:38:51	0:00:00
6	itlb	PASS	1	0	100%	13:38:52	13:38:58	0:00:07
7	regfile2	PASS	1	0	100%	13:38:59	13:39:05	0:00:07
8	regfile3	PASS	1	0	100%	13:39:06	13:39:12	0:00:07
9	enet_sd_lpbk	PASS	1	0	100%	13:39:13	13:39:23	0:00:24
10	spd_checksum	PASS	1	0	100%	13:39:24	13:39:25	0:00:00

6.3.1 Regression Tests

These tests are a predetermined groups of tests. They are used to test a given set of functionality and are the equivalent of regression scripts. These tests typically consist of one or more groups of tests selected with test qualifiers and run with `board.run_test()`, but it can also include component self-tests or other commands to verify a set of functionality; `board.gx_regress()` is an example of a regression test.

6.4 Board Devices and Self Tests

6.4.1 Self-Tests

Self-tests verify the functionality of a chip component interface (like an rshim or enet) or in the `board_devices` dictionary. They are invoked by running `rshim.test()`, `enet.test()`, or the names of other hardware interfaces or `board.board_device[name].test()`.

`board.test()` executes all tests in the `board_devices` dictionary.

The purpose of these tests is to show that a chip interface or board device is alive and responsive.

These tests are not within the `tests[]` dictionary. They belong to hardware interfaces (shown with the `dir()` command or for devices in the `board.board_devices` dictionary). They are not selectable with test qualifiers to run using the command `board.run_tests()`. The command `board.test()` runs all available self-tests for devices in the `board.board_devices` dictionary. Chip component interfaces are not added as board devices, and their self tests are executed by running `chip.alive()`, which is to the chip as `board.test()` is to the board.

6.5 Logging

When BTK is started, the option `--log <directory>` or `-l <directory>` creates a log directory to log many BTK commands and test output. As an example, `tile-btk -l.` starts BTK with a log file in the current directory. The base BTK log file created is `<directory>/tilera_logs/btk*.log`. All logs created by BTK are in the `<directory>/tilera_logs` directory. If auto-discovery is used to determine the board attributes, the Serial Number of the board is included in the log name. This allows for logs from different boards to be easily identified.

Subdirectories under `tilera_logs` can also separate log files by hardware type.

The TILE-Gx™ BTK uses Python's *logging* module. Refer to Python documentation on this module for more information.

6.5.1 What Is Logged

The main BTK log, `btk.log`, is part of the *board* interface. This log captures any text written with `board.log.<level>()` and also prints the same text to STDOUT. For example, `board.log.info("This is the most common logging level")` prints to `btk.log` and to the console.

The default level of details to print to the BTK log is *debug* and *info* to print to the console.

Possible logging levels are *debug*, *info*, *warning*, *error*, and *critical*. BTK also logs everything printed to STDOUT by the python `print` method to the same log files used by the logging module; it does so by employing a custom `sys.stdout` class.

Using `print` has the disadvantage of not using any features offered by the logging module, but it does allow more freedom in formatting displayed output.

6.5.2 Setting Logging and Print Levels

The logging level that appears on the screen or in a test log is set with the commands `board.set_print_level(<level>)` and `board.set_log_level(<level>)`. Possible levels are *debug*, *info*, *warning*, *error*, and *critical*. For example,

`board.set_print_level('warning')` results in `board.log.<level>` statements of *warning* or higher printed to the console. *Debug* and *info* levels do not appear on the console, but they will still be logged in `bt* .log`.

If `board.set_log_level(<level>)` is used instead, only messages of the level indicated or higher are logged. Additionally, `sys.stdout.display` can be set to `True` or `False`. If `False`, no output is displayed within the BTK console; output is still displayed to the log file(s).

6.5.3 Test Logs

Unless otherwise directed, the results of `board.run_tests()` are logged in `run_tests.log`. Regression tests follow the same naming convention. Thus, `board.gx_regress()` enables results to be captured in the `gx_regress.log`.

Test details are stored in the log beginning with `bt* .`

6.5.4 Log Organization

BTK logging was enhanced beginning in MDE 4.1.6. To know if your version of BTK uses the new logging mechanism, look where BTK logs are stored. Originally, logs were in `... | /logs/`, but have not been moved to: `... | /tilera_logs/`.

These are the major changes:

1. There are now multiple log files; one per node plus a system log file if appropriate. The default log for `sys.stdout` is either the **System** or **Node 0**, depending on startup options.
2. `tilera_logs/` is the top level BTK logging directory, appended to whatever is specified by the `--log` option.
3. `sys.stdout` points to a custom `STDOUT` class. This class is threading aware and output directed to `sys.stdout` is multiplexed between different log files, depending on a given **Nodes** mapping to running threads.
4. A `set_context` method automatically redirects output to the appropriate log file for a given node when using the `run_tests()` method.
5. `CustomConsole` is derived from `InteractiveConsole` that allows proper functioning of `Interactive Console` with a redirected `stdout`, as mentioned above.

Logs are organized under the `/tilera_logs/` directory categorized by **System** or **Board type** and **board part number**. The new logging features are most important for the BTK session where there are many **Nodes**. This does not apply to most users, but when it does, this feature is essential for directing output appropriate to a given **Node** to the correct log. When testing multiple nodes at once, many tests execute simultaneously (one per node) using Python's threading capabilities.

Instead of only one `bt* .log` file as before, logs names are now based on their node designation for units with multiple TILE-Gxs. Examples are `bt*_node_1_ROK-01058-00012` or `BTK_node_4b`. If the serial number of the system is known, as it with "Auto-discovery", the serial number is included in the log name.

Listing 6-1. Example Log Directory

Directory:

```
.../tilera_logs/450-00038-80G/402-00047-01
```

Files:

```
btk_node_1a_ROK-00047-00009.log  comp.log  running_comp.log  run_tests.log
```

Description:

In this example, the board class was found using Auto-discovery, which allowed for a log directory to be created that includes the type of system or board class (450-00038-80G) and the part number of the board (402-00047-01). This is a multi-node platform that includes the node designation (1a) and the serial number of the discovered board (ROK-00047-00009) included in the btk log name.

6.5.4.1 Suppressing Output to the Display

In some cases, especially when running multithreaded tests, users want to suppress prints to the display (prints and log statements go to logs only).

- `sys.stdout.display = False` suppresses prints to the display, but dots (.....) are printed to indicate that tests are in progress.
- `sys.stdout.display = True` causes all output to be printed to the screen (assuming that the log verbosity is high enough for it to be displayed).

CHAPTER 7 SETTING UP YOUR BOARD AND CONFIGURING YOUR BIB

7.1 Introduction

When you create your own board, you need to configure a Board Information Block (BIB). This chapter describes how to set up your board and configure a BIB for it. For systems that include multiple chips, multiple BIBs should be created.

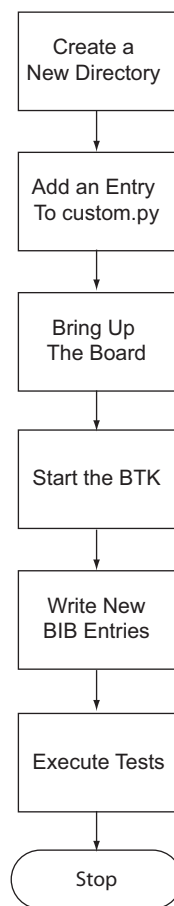


Figure 7-1. Board Creation Flowchart

For information about the structure of the BIB, refer to *Tilera Board Information Block Format* (AN040) in the `doc/html/board-info.html` file in the MDE Document Index.

Refer to the *Hardware Design Guidelines for TILE-Gx36, TILE-Gx16, and TILE-Gx9 Processors* (WP006) or *Hardware Design Guidelines for TILE-Gx72 Processors* (WP007) for information about designing your own board using a TILE-Gx™ processor.

7.2 Configuring the BTK to Work with a Custom Board

1. Create a new (custom) directory that contains all of your customizations, and copy the contents of `$TILERA_ROOT/lib/tile-btk/example_custom` into that directory.

2. Add an entry to `custom.py` for your new board file to customize your board.

a. `import my_new_board` To add a new `my_new_board.py` file.

b. `my_class = my_new_board.NewBoardClass`
 within `parts2class` method to default to your board class with Auto-discovery. Note that this refers to the `parts2class` method in `custom.py`.

3. Create a new board file, using the contents of `example_board.py` as a reference.

a. `def reset` To customize board reset instructions.

b. `def create_bib` To customize the BIB entry.

Define the BIB as shown in the example:

```
def create_bib(self, serial=None, mac_factory=None):
```

```
    bib = board_info.Bib()

    if mac_factory is None:
        mac_factory = utils.MacFactory()
    bib.append(
        board_info.board_part_num(
            instance=0,
            part_num="my-part-number",
        )
    )
```

c. `def __init_board` To use the method for any custom board initialization.

d. `def __add_tests` To add any new tests specific to this board. In this step, you add desired board devices (not SROM), often I²C.

- e. Add subclasses to your new board file that extend the functionality of board base class methods found in `board.py`.

1) `create_access_points()` To add a new `chip_access` mechanism.

2) `connect_to_chip()` To connect to a new `chip_access` mechanism.

Note: You rarely need to perform this step.

5. From the host, start BTK specifying your custom board entry:

```
$TILERA_ROOT/bin/tile-btk --dev <device> --custom <your-custom-directory>
```

6. Start BTK and write new BIB Entries for your New Board. From the `btk` command line, call the `create_bib` function you defined in your board file to get a BIB object.

```
btk> b = board.create_bib()
```

To write that BIB object to an I²C ROM:

```
btk> b.write(board.chip, (0, i2c_rom_address))
```

Replacing “`i2c_rom_address`” with the specific value for your board. A typical value is “`0xa0`”.

To write the BIB to a SPI ROM:

```
btk> b.write(board.chip, (0, -1))
```

Note: A new `--chip` BTK startup parameter has been added, which allows specification of the chip type (SKU) to be used. If the chip has not been specified, Auto-discovery detects the correct chip type, even if the `--board` parameter is used.

7. Exit and re-enter the `btk` without the `--board` option (to verify the BIB is written correctly).
8. Read the BIB using `board.bib.dump()`.
9. Execute tests. Follow the procedure in [Running BTK Tests](#) on page 31 for executing tests.

7.3 Registering a Board Class

In the following example, the Board Class is registered:

```
#
# This is at the end of your custom Board Class file.
# "my-part-number" represents a customer defined part number.
#

# Add board class to dictionary for lookup
board.register("example", BoardExample, "Example_Board")

part_num_list = ("my-part-number",)
board.register_parts(part_num_list, BoardExample)
```

1. Register the Board Class to be selectable with the `--board` parameter at startup. For example:

```
./btk --dev usb0 --board example
```

Two registration methods should be added after the Board Class is defined. The first method adds the board class to the board dictionary:

```
board.register("example", BoardExample, "Example_Board")
```

2. Register the Board Class to be auto-discovered with the custom `parts2class()` method. Starting BTK with this option alone finds the Board Class, as shown in the following example:

```
./btk --dev usb0
```

The second method registers the Board Class for Auto-discovery:

```
part_num_list = ("my-part-number",)
board.register_parts(part_num_list, BoardExample)
```

7.4 Adding New SROMs

Near the beginning of the example custom Board Class file include the following sample SROM dictionary, as shown in the example. Add new SROM devices to this table for custom boards.

```
import srom

custom_srom_table = {
    # ID value      Name      page      sector      srom
    #              size      size      size
    # Dictionary is EMPTY unless custom entries are added as below
    0x20BA191500: ("EXAMPLE SROM1", 256, 0x10000, 0x1000000),
    0x20BA191501: ("EXAMPLE SROM2", 256, 0x10000, 0x1000000),
}

# Register our custom SROM
table.srom.register_custom_srom(custom_srom_table)
```

To display all available SROM devices at the btk prompt, import the SROM as shown in the example:

```
btk> import srom
btk> for device in srom.device_table:
...     print srom.device_table[device]
...
('Numonyx M25P128', 256, 262144, 16777216)
('Macronix MX25U12835F', 256, 65536, 16777216)
('EXAMPLE SROM1', 256, 65536, 16777216)
('Spansion SF25FL064A', 256, 65536, 8388608)
('Spansion SF25FL128Pxxxxx01x', 256, 262144, 16777216)
('Winbond W25Q128BV', 256, 65536, 16777216)
('ST Micro M25P64', 256, 65536, 8388608)
('Spansion SF25FL128Sxxxxx01x', 256, 262144, 16777216)
('Macronix MX25L12{845E,805D}', 256, 65536, 16777216)
('EXAMPLE SROM2', 256, 65536, 16777216)
('Micron N25Q256A', 256, 65536, 33554432, 200, 197)
('Spansion SF25FL256Sxxxxx00x', 256, 65536, 33554432, 22, 23)
('Spansion SF25FL512S', 512, 262144, 67108864, 22, 23)
('Numonyx N25Q128', 256, 65536, 16777216)
('Numonyx M25P64', 256, 65536, 8388608)
('Spansion SF25FL128Pxxxxx00x', 256, 65536, 16777216)
('Spansion SF25FL128Sxxxxx00x', 256, 65536, 16777216)
```

7.5 Adding a New PHY

1. Check to see if the base class has already been defined. `ENET_PHY` is the base class that most PHY's will use. However, there might already be a family base class defined. For example, the `BCM8747_PHY` class inherits from the `BCM87X7_PHY` class. If the base class already exists, it is listed in the `enet_phy.py`. Import the best base class for your device into the desired file in your custom directory.
2. Add an entry to the `enet_phy_discovery()` by running a registration method for your new PHY class. The new PHY class must have a unique id assigned within the class.

```
register_phys(EXAMPLE_PHY.id, EXAMPLE_PHY)

class MV88E1111_PHY(ENET_PHY):
    """MV88E111 Gbe PHY"""
```

```

id = 0x0cc20141

def __init__(self, board, mac_device, phy_addr, reset_func=None):
    super(MV88E1111_PHY, self).__init__(board=board,
                                         mac_device=mac_device,
                                         phy_addr=phy_addr,
                                         reset_func=reset_func)

    self.type = 'MV88E1111'
    self.speed = '1G'
    self.supported_speeds = ['10M', '100M', '1G']
    self.duplex_mode = 'FULL_DUPLEX'
    self.supported_duplex_modes = ['HALF_DUPLEX', 'FULL_DUPLEX']
    self.loopback = 'DISABLED'
    self.an_ena = 'DISABLED'
    self.config_enabled = False

```

Notes about bringing up the Ethernet PHY:

- The BTK does not automatically discover or instantiate objects for the PHYs by default.
- PHY(s) to be instantiated are described in the BIB, added with the `bib.append()` method.
- Successful MDIO reads of PHY vendor ID registers via the TILE-Gx mPIPE's MDIO interface create PHY objects added to a list under the BTK called `enet.phy_devices`.
- Bringing up the PHY assumes a working TILE-Gx™, mPIPE, and memory subsystem.

7.6 Adding Temperature Sensor Classes

Adding I²C Device Classes and classes for functionality groups can be useful, but in fact creating a BIB entry might be all that is required. If you want to add a Temperature Sensor Class to BTK, add the new sensor to the custom directory using sensors in `temperature_sensors.py` as a reference.

Use the file `environmental.py` to view methods that can be used to add a `TempSensor` class in `__create_board_devices()`.

Temperature Sensor Classes inherit characteristics from two classes: `I2cDevice` and `TempSensorBase`.

`self.description` is set to `Temp Sensor`. This is necessary if adding a `TempSensor` class to the Board Class. This is how the `TempSensor` class discovers I²C devices that are temperature sensors.

```

class Lm95235(I2cDevice, TempSensorBase):
    """This provides information about the Lm952325"""

    def __init__(self, board, name, address, channel, bus):
        super(Lm95235, self).__init__(board, name, address, channel, bus,
                                      address_bytes=0)

        self.description = 'Temp Sensor'
        # This entry continues in temperature_sensors.py

```

7.7 Advanced Custom Configurations

For cases where multiple custom boards or the same board with multiple TILE-Gx variants (such as with cryptography or without cryptography, TILE-Gx16 or TILE-Gx36), additional customizing is not necessary.

7.7.1 Multiple or Undefined Chip Variants

The bringup of custom boards is simplified by using auto-discovery. The auto-discovery function identifies the chip's parameters. If the auto-discovery function fails, the default chip class defined in the custom board class is used.

7.7.2 Multiple Custom Board Classes

The easiest way to add multiple custom board classes involves creating a different custom directory for *each* board class. If multiple custom directories are used, you do not need to create a custom `parts2class` method for differentiating different board types when using Auto-discovery.

However, a single directory can also be used to support multiple board classes, which entails some additional steps. First, the board must have a BIB entry in an SRAM or I2CROM, and the part number field in the BIB should be utilized. While it is possible to detect multiple board classes without using the part number field, the `parts2class()` method in `custom.py` needs to be customized further to find some attribute to denote fields in the BIB that could be used to differentiate one board type from another.¹

1. The `parts2class()` method in `custom.py` is a simplified version of what Tileria® uses to differentiate platforms produced by Tileria through Auto-discovery.

CHAPTER 8 WORKING WITH THE DDR3 INTERFACE

This chapter describes how to configure DDR3 memory interfaces in the following sections:

- [Setting Up the Board Information Block](#)
- [Configuring a Memory Interface and Running a Test](#)
- [Using Common Commands to Specify Memory Configuration Options](#)

For more information, refer to *Application Note: Bringing Up a TILE-Gx DDR3 Interface Using the BTK* (AN045).

8.1 Setting Up the Board Information Block

Before using a DDR3 interface, it is necessary to program the Board Information Block (BIB) for the board. See [Chapter 7. Setting up your Board and Configuring your BIB](#) for details on programming the BIB and refer to *Tilera Board Information Block Format* (AN040) in the `doc/html/board-info.html` file in the MDE Document Index for details on how to specify DDR3 information in the BIB.

For interfaces with DIMM sockets, the board information block provides information about how many DIMM sockets there are, how chip selects are connected to each socket, and which I²C address can be used to access the Serial Presence Detect (SPD) ROM on a DIMM in each socket.

For interfaces with RAMs soldered to the board, the board information block provides information about the RAMs with an SPD-format item.

8.2 Configuring a Memory Interface and Running a Test

Memory interface configuration is done by running either a memory interface test or a test that will be loaded into off-chip memory. All of these tests configure one or both memory interfaces before running the test.

To begin working with the memory interfaces of the chip, the following tests are included in the BTK.

<code>config_mem</code>	Configures one interface (no real “test” after that).
<code>bringup_memory</code>	Configures memory and then runs a basic tile-based test of memory.
<code>mem_stress</code>	Configures memory and then runs a more stressful tile-based test of memory.

These tests all work with just one DDR3 interface, which is specified via the `primary_mshim` attribute of the test. For example, to configure DDR3 interface 1:

```
btck> tests["config_mem"].primary_mshim = 1
btck> tests["config_mem"].run()
```

At a high level, the memory interface configuration process involves the following steps.

1. The DDR clock is configured to the specified operating speed.
2. The DDR voltage is brought up to a specified voltage.
3. The Board Information Block (BIB) is queried to discover the characteristics of the DIMMs/RAMs attached to the interface and the board itself.
4. The DDR interface controller is configured based on those RAM characteristics.
5. The DDR interface and RAMs/DIMMs are brought out of reset and set up.
6. A procedure, which will measure the delays of interface signals and then configure the compensating delays in the controller, is initiated.
7. Configuration is finalized and the off-chip memory is initialized.

8.3 Using Common Commands to Specify Memory Configuration Options

Generally, the memory configuration process is controlled by manipulating attributes of the memory shim (`msh`) object and then running a test that performs the configuration. For example, to control the operating speed of the interface (specified in transfers/second):

```
btck> msh[0].clkgen_pll_freq = 800000000
btck> tests["config_mem"].primary_mshim = 0
btck> tests["config_mem"].run()
```

To increase the output displayed during the interface configuration process:

```
btck> msh[0].verbose = True
```

To control the DDR3 operating voltage:

```
btck> board.set_ddr_voltage(1.35)
```

To display the SPD information being used for memory configuration:

```
btck> msh[0].display_spds()
```

To disable the configuration/use of error correction (ECC):

```
btck> msh[0].enable_ecc = False
```

The following methods of the mshim object can be used *after an interface has been configured* to probe memory.

<code>mem_write(address, data, size)</code>	Write a memory location.
<code>mem_read(address, size)</code>	Read a memory location.
<code>mem_dump(address, length)</code>	Read and display a range of memory locations.
<code>test()</code>	Run a simple memory test (not as thorough/stressful as tests described above).

For example, after configuring memory interface 0:

```

btk> msh[0].mem_write(0, 1234)
btk> msh[0].mem_read(0)
1234L
btk> msh[0].mem_write(0, 0x1234)
btk> msh[0].mem_dump(0)
    07 05 03 01 0f 0d 0b 09 17 15 13 11 1f 1d 1b 19
      06 04 02 00 0e 0c 0a 08 16 14 12 10 1e 1c 1a 18
0000000000 0000000000001234 eeeeeeeeeeeeeeee dddddddddddddddd cccccccccccccccc
      4.....
0000000020 bbbbbbbbbbbbbbbb aaaaaaaaaaaaaa 9999999999999999 8888888888888888
      .....

```

There are many other controls over the configuration process, but they are beyond the scope of this document and typically used only for diagnostic/debug purposes. Their use is covered in Application Note: *Bringing up a TILE-Gx DDR3 Interface Using the BTK* (AN045).

APPENDIX A: COMMANDS MATRIX TABLE

This chapter provides a table of legacy TILE64™/TILEPro™ BTK commands, the corresponding new TILE-Gx™ commands, and newly-added commands.

Table A-1. BTK Commands Matrix Table

TILE64 / TILEPro Commands	New TILE-Gx Commands	Descriptions
<code>tlr.test_init()</code>	<code>chip.reset()</code>	Reset the processor.
<code>tlr.tile_init()</code>	<code>tests['test_name'].run()</code>	Reset the processor and load standard “phase 1” boot code.
<code>tlr.mem_init()</code>		Reset the processor, load “Phase 1” boot code, and configure all memory interfaces that are being used.
<code>run_test("testname")</code>		Run a test.
<code>utils.add_new_test("testname", (list_of_source_files))</code>	New tests are added in the subclass for a new board, not at the command line.	Add a new test to the test catalog.
<code>boot.set_mshim(interface_num)</code>	Settable on a per test basis with: <code>tests["testname"].primary_mshim = interface_num</code>	Select the memory interface behind which “level 2” test code will be loaded. This will also be the interface that will be tested in “Phase 1” tests that access memory.
<code>config.cclk_freq = freq</code>	<code>chip.set_core_clock_rate()</code>	Set core clock frequency immediately. Once it is set, it stays in effect until it is changed again.
	<code>msh[interface_num].config_pll()</code>	Change frequency upon execution.
	<code>msh[].config_clocks()</code>	This is an automated that will change clocks back to default settings upon execution.
<code>config.msh_list[interface_num].freq = freq</code>	<code>msh[interface_num].clkgen_pll_freq = freq</code>	Select the memory interface speed, set next time memory is configured.
<code>rshim.test()</code>	<code>rshim.test()</code>	Test ability to access Rshim device registers.

Table A-1. BTK Commands Matrix Table (continued)

TILE64 / TILEPro Commands	New TILE-Gx Commands	Descriptions
<code>rshim.write(channel, address, data)</code>	This is now a multi-step process: <code>rshim.regs.reg-name.read()</code> <code>rshim.regs.reg-name.field_name = data</code> <code>rshim.regs.reg-name.write()</code>	Write an Rshim device register.
<code>rshim.read(channel, address)</code>	<code>rshim.regs.reg-name.dump()</code>	Read an Rshim device register.
<code>tlr.msh_diag_wr(interface_num, address, data)</code>	<code>msh[interface_num].mem_write(addr, data)</code>	Write 64 bits of data to given memory interface and physical address. If running on a TILEPro processor, the physical address will be modified accordingly if address hashing or striping mode is enabled.
<code>tlr.msh_diag_rd(interface_num, address)</code>	<code>msh[interface_num].mem_read(addr)</code>	Read 64 bits of data from given memory interface and physical address. If running on a TILEPro processor, the physical address will be modified accordingly if address hashing or striping mode is enabled.
<code>display_spd()</code>	<code>msh[interface_num].display_spds</code>	Display the current SPD configuration values.
<code>display_msh_config()</code>		Display the current memory controller configuration values.
<code>tlr.dump_msh_regs(shim_x, shim_y)</code>	<code>msh[interface_num].dump()</code>	Old command: Display the current values of the configuration registers of a memory controller configuration. New Command: This will dump ALL registers in a mshim.
<code>debug.check_all_tiles_L2(physical_address)</code>	<code>chip.find(physical_address)</code>	Probe all tiles on the chip to see if the physical address is present (valid).
<code>debug.print_pcs()</code>	<code>chip.print_pcs()</code>	Print the PCs of all of the tiles.
<code>debug.print_pcs_and_lrs()</code>	<code>chip.print_pcs_and_lrs()</code>	Print the PCs and Link Registers of all the tiles.
<code>debug.print_network_state(<network-name>)</code>	<code>chip.print_network_state(<network-name>)</code>	Print a picture representing the state of one of the mesh networks.
<code>debug.print_tile_physical_memory(x_coordinate, y_coordinate, physical_address)</code>	No longer supported.	In the given tile, probe the Level 1 and Level 2 caches to see if the physical address is present (valid), and print the data if it is.
<code>debug.print_tile_state(x_coordinate, y_coordinate)</code>	<code>tile(x, y).dump()</code>	Print the PC and registers of a specific tile.

Table A-1. BTK Commands Matrix Table (continued)

TILE64 / TILEPro Commands	New TILE-Gx Commands	Descriptions
<code>debug.read_spr(x_coordinate, y_coordinate, SPR_index)</code>	<code>tile(x,x).sprs.reg-name.dump()</code>	Read an SPR in a tile.
<code>rshim.i2c_read(slave_address, address, data, [use_8_bit_addressing])</code>	<code>rsh_i2cm[interface_num].read(slave_address, dev_offset, length)</code>	Read a byte to an I ² C device.
<code>rshim.i2c_write(slave_address, address, data, [use_8_bit_addressing]) X1</code>	<code>rsh_i2cm[interface_num].write(slave_address, dev_offset, data)</code>	Write a byte to an I ² C device.
	<code>i2cmX.show_all()</code>	See all the I ² C devices on a given interface. Read and Write to those devices without worrying about setting muxes in between. There is also a higher level I ² CM interface i2cm0, i2cm1, and i2cm2
<code>rshim.srom_read(address, num_words)</code>	<code>rsh_srom.read(offset, length)</code>	Read some number of xx-bit words from the SPI SROM.
<code>rshim.srom_write(address, num_words)</code>	<code>rsh_srom.write(offset, data)</code>	Write some number of xx-bit words from the SPI SROM.
<code>help()</code>	<code>help()</code>	The help command is still callable on any object, for example: • <code>help(chip)</code> • <code>help(board)</code> • <code>help(tile(0, 0))</code> • <code>help(rshim)</code> • <code>help(msh)</code>

APPENDIX B: BOOTING THE MDE OVER UART

B.1 Introduction

This section explains how to boot the MDE on a TILE-Gx™ processor using a UART (serial) port. Typically we do not recommend this; using the USB or PCIe interfaces is much faster and more robust. However, some boards do not have access to those interfaces, and in those cases serial boot might be required.

As of MDE 4.0.1, the BTK and the MDE support serial boot in two configurations:

1. Dual-UART boot. In this configuration, one of the TILE-Gx UART ports is configured in protocol mode, and is used to inject the boot stream. The other port is not in protocol mode, and is used for the system console. This is the preferred configuration if both ports are available, since it is slightly easier to use.
2. Single-UART boot. In this configuration, one of the TILE-Gx UART ports is configured in protocol mode, and is used both to inject the boot stream, and to relay early console output; the BTK polls registers within the Rshim periodically during the boot to accomplish this. Once the boot stream has been consumed, that UART changes to non-protocol mode and becomes the system console. This is slightly more difficult to use (since you must exit the BTK and start `tile-console` once the system has finished booting), but is required when only one port is available.

B.2 Preparation

There are several things you will need to do before attempting your first UART boot.

1. Build the bootrom.

When booting over USB or PCIe, you typically let the `tile-monitor` command automatically construct the boot stream “on the fly” as part of the boot process. However, when booting over UART, you will be using the BTK, and the BTK does not have the ability to do this; you must pre-construct the boot stream and save it to a file that will be used by the BTK. The easiest way to do this is with `tile-monitor`’s `--create bootrom` argument, as follows:

```
example$ tile-monitor --no-dev --create-bootrom <filename>
```

where `<filename>` is the name of the output file.

You can also use the `tile-mkboot` command directly, but if you do so, you must specify the `--compress` argument. An uncompressed bootrom is about twice the size of a compressed one, and takes about twice as long to inject it into the target TILE-Gx processor. While this is not an issue over USB or PCIe, it causes problems over the UART.

2. Set up tile-dev configuration files.

The next step is to modify or create a tile-dev configuration file for the target system. This step is optional, but recommended, since it eliminates the need to specify explicit tty device names when running the BTK or tile-console.

To do this, on the host system, create a file called `/etc/tilenet/<target>.conf`, containing the following three lines:

```
label=<target>
serial=<console-device>
serial_btk=<btk-device>
```

where:

<code><target></code>	is an arbitrary name for the target system.
<code><console-device></code>	is the host device pathname for the serial port connected to the TILE-Gx UART which will be used for the console.
<code><btk-device></code>	is the host device pathname for the serial port connected to the TILE-Gx UART which will be used for injecting the boot stream.

If you are using the single-UART configuration, then `<console-device>` and `<btk-device>` are the same, and you can omit the `serial_btk` line.

As an example, assume we have a TILE-Gx system that we will call “bob”. You are using the dual-UART setup. You have connected TILE-Gx UART 0 (console) to the host’s first serial port, and TILE-Gx UART 1 (boot) to the host’s second serial port. Your config file would then look like this:

```
example$ cat /etc/tilenet/bob.conf
label=bob
serial=/dev/ttyS0
serial_btk=/dev/ttyS1
example$
```

Note that it is also possible to add the `serial` and `serial_btk` lines to an existing tile-dev configuration file, for instance, one configured for booting via USB or PCIe.

3. Set up the hardware.

First, connect your host system to the target platform via a serial cable (or two serial cables, for the dual-UART configuration). Before you can boot over UART, you must also ensure that two things are true:

- a. The TILE-Gx UART port over which you will boot is configured for protocol mode (i.e., the `CONFIG_UART[0]` or `CONFIG_UART[1]` pin, as appropriate, is high).
- b. The TILE-Gx master boot interfaces (SPI ROM and I²C EEPROM) are disabled (i.e., the `CONFIG_ROM[1:0]` pins are both low).

The first is required so that the TILE-Gx chip actually obeys the commands you sent it via the UART; the second is required because if you try to boot the chip via UART while it is simultaneously trying to boot from, say, SPI ROM, neither boot will succeed. On Tileria® board products, these settings can be changed via DIP switches or jumpers; see the board hardware documentation for details.

4. Test connectivity.

Once the target chip has been configured, you can test connectivity. Reset the chip (either turn the power off and back on, or, if the system has a reset button or switch, use it). Then, run the BTK, specifying the name of the configuration file and the `--serial` argument, as follows:

```
example$ tile-btk -d <target> --serial
```

where `<target>` is the name used for the configuration, as above. If you did not create a configuration file, you can specify the serial port explicitly instead, as follows:

```
example$ tile-btk --serial -serial-device <btck-device>
```

where `<btck-device>` is the name of the host serial device, as above.

In either case, if you are successfully communicating with the target, the BTK should be able to dynamically probe the chip properties, and display a BTK prompt. That will look something like this:

```
example$ btk -d target --serial
/tmp/tile-btk-ttkUiO-user/logs created as temporary log directory
tile-btk: Serial device for chip access is /dev/ttyS0

Exploring chip to find devices and fabric dimensions
Discovered tile fabric dimensions are 6 x 6
Discovered rshim coordinates are 1,-1
Discovered device table
Type           Instance      X      Y
----
CRYPTO           1          6      0
CRYPTO           0         -1      0
RSHIM           0          1     -1
DDR3            1          6      1
DDR3            0         -1      1
USBS            0          2     -1
MPIPE           0          2      6
COMPRESSION     0          3     -1
COMPRESSION     1          3      6
GPIO            0          4     -1
TRIO            0          5      6
Initialized TILEmpowerGx36 board
Tileria TILE-Gx Board Test Kit
btk>
```

If you want to perform a more extensive test, you can run `rshim.test()`:

```
btk> rshim.test()
Pass 0 progress:
0% |-----| 100%
=====
```

```

Finished pass 0
True
btk>

```

If things do not work, the BTK might stop after the “Exploring chip” line, or you might see a message like “ERROR: UART is not responding”, or your `rshim.test()` call might fail. If any of these events occur:

- Check the wiring of your serial port cable.
- Ensure that the host is not waiting for any kind of modem control signals on that serial port. Try doing “`stty -F <btk-device> -a`” and make sure that the output includes “`-clocal -crtsets -cdtrdsr`”.
- Make sure no other processes on the host are using that serial port.
- Make sure the target TILE-Gx processor is really being reset.

B.3 Booting

After you have completed the preparatory steps listed above, it is easy to perform a boot, as follows:

1. Run the BTK (if you have not already), as described above.

```
example$ tile-btk -d <target> --serial
```
2. If you are using both UARTs, then in another window, connect to the console via the `tile-console` command, for example:

```
example$ tile-console --dev <target>
```

where `<target>` is the name used for the configuration, as above. If you did not create a configuration file, you can specify the serial port explicitly instead, as follows:

```
example$ tile-console --serial <console-device>
```

If you are using only one UART, skip this step for now.

3. Type the `chip.boot_uart()` command at the BTK command line. This command takes the following arguments:
 - `filename`: Name of the bootrom file. This is a Python string, and thus is enclosed in quotes. It can be an absolute or relative pathname.
 This is the only required parameter.
 - `speed`: baud rate to use. By default, this is 115200, but on some systems you can successfully use 230400 or even 460800.
 If you have problems booting, try setting a slower speed.
 - `port`: which TILE-Gx UART is being used (0 or 1). This is only relevant when speed is not 115200; the default is 0. Note that this has nothing to do with the name or number of the host serial port; this is the number of the serial port as viewed by the TILE-Gx.

- `boot_on_console`: if `True`, the TILE-Gx UART we are using to inject the boot stream is the same one that the hypervisor will use as the system console. Thus, this will be `True` if you are using the single UART configuration, and `False` if you are using the dual-UART configuration. By default, this is `True`.
 - `hash`: if nonzero, the BTK will print a # for each hash bytes of bootstream injected. This is useful for confirming that the boot is progressing. By default, hash is set to print a # every 32 K bytes.
4. Wait for the boot to complete. The BTK will give you an estimate of the time for the boot stream to be injected; depending upon the serial port baud rate and the size of your bootrom this could easily be 10 minutes or even longer. If you are using the single-UART configuration, the BTK will print any console output directly in the BTK window. In the dual-UART configuration, console output will appear in the `tile-console` window started earlier.
 5. Finally, once the boot is done, you can interact with the booted system. If you are using the single-UART configuration, exit the BTK, and start `tile-console` on the console serial port. If you are using the dual-UART configuration, you need not exit the BTK, and can instead switch to the `tile-console` program you started earlier.

B.4 Booting Examples

1. Here is an example of a boot in a single-UART configuration.

```
example$ btk -d single-target --serial
/tmp/tile-btk-phNEJX-user/logs created as temporary log directory
tile-btk: Serial device for chip access is /dev/ttyS8
```

```
Exploring chip to find devices and fabric dimensions
Discovered tile fabric dimensions are 6 x 6
Discovered rshim coordinates are 1,-1
Discovered device table
```

Type	Instance	X	Y
-----	-----	-	-
CRYPTO	1	6	0
CRYPTO	0	-1	0
RSHIM	0	1	-1
DDR3	1	6	1
DDR3	0	-1	1
USBS	0	2	-1
MPIPE	0	2	6
COMPRESSION	0	3	-1
COMPRESSION	1	3	6
GPIO	0	4	-1
TRIO	0	5	6

```
Initialized TILEmpowerGx36 board
```

```
Tilera TILE-Gx Board Test Kit
```

```
btk> chip.boot_uart("serial.bootrom")
```

Your bootrom contains 7274960 bytes; it will take around 810 seconds just to transfer the boot data via UART. Time spent running POST, etc. is in addition to this, so the overall boot may take longer. Note that console input is not supported while booting over UART, so do not type at any prompts. Once the boot is complete you can exit the BTK and interact with the booted system using `tile-console`.

```
Setting core frequency to 1200 MHz (m = 47, n = 4, q = 1, r = 2)
VID control for core voltage set to 1.05000V (0x1a)
VID control for DDR voltage set to 1.50000V (0x02)
```

```

##
Type t in 3 seconds to run thorough POST tests, q to run quick tests...
Thorough POST tests will be run.

msh0 0123 msh1 0123
#####(0,0) Tilera Hypervisor, version 4.0.1.141599 2012-04-09 18:25:53

#####

#####

(0,0) Built by user from /opt/TileraMDE-4.0.1.141599/tilegx/tile/etc/hvc/
vmlinux.hvc, /opt/TileraMDE-

4.0.1.141599/tilegx/tile/boot
Initializing cgroup subsys cpuset
Initializing cgroup subsys cpu
Linux version 2.6.38.8-MDE-4.0.1.141599 (support@tilera.com) (gcc version 4.4.6
GCC) ) #1 SMP Thu

Apr 5 22:07:55 EDT 2012
32720MB memory available.
PCI: Searching for controllers...
.
. (Core of Linux boot output omitted)
.
Starting tempmon daemon.
Starting dropbear sshd.
Starting shepherd.
tempmond: thresholds: chip 100C/110C, board 100C/110C; poll 5s

Please press Enter to activate this console.

Boot took 942 seconds.
btk>

```

2. And here is an example of a boot in a dual-UART configuration. Note that you do not see any console output, since that all went to the tile-console window instead.

```

example$ btk -d dual-target --serial
/tmp/tile-btk-xNNioZ-user/logs created as temporary log directory
tile-btk: Serial device for chip access is /dev/ttyS11
Exploring chip to find devices and fabric dimensions
Discovered tile fabric dimensions are 6 x 6
Discovered rshim coordinates are 1,-1
Discovered device table

```

Type	Instance	X	Y
-----	-----	-	-
CRYPTO	1	6	0
CRYPTO	0	-1	0
RSHIM	0	1	-1
DDR3	1	6	1
DDR3	0	-1	1
USBS	0	2	-1
MPIPE	0	2	6
COMPRESSION	0	3	-1
COMPRESSION	1	3	6
GPIO	0	4	-1
TRIO	0	5	6

Initialized TILEmpowerGx36 board
 Tiler TILE-Gx Board Test Kit
 btk> chip.boot_uart("serial.bootrom", speed=230400, port=1, boot_on_console=False)
 Setting core frequency to 1200 MHz (m = 47, n = 4, q = 1, r = 2)
 VID control for core voltage set to 1.05000V (0x1a)
 VID control for DDR voltage set to 1.50000V (0x02)
 Successfully changed UART speed to 230400 baud.

Your bootrom contains 7274960 bytes; it will take around 398 seconds just to transfer the boot data via UART. Time spent running POST, etc. is in addition to this, so the overall boot may take longer.

```
#####
#####

#####
#####

#####
Boot took 395 seconds.
btk>
```

B.5 Using the BTK after Boot

When using the dual-UART configuration, the UART that was used to inject the boot stream will still be in protocol mode once the boot is complete. This means that you can continue to use the BTK to do debugging or diagnostic operations. However, in the single-UART configuration, once the system has booted, the UART is no longer be in protocol mode, so those operations will be unavailable.

The hypervisor provides a console escape sequence that puts the console UART back in protocol mode to enable debugging. To simplify the process, the BTK provides a command `-- uart.start_protocol_mode()` — that issues this escape sequence, and then reconfigures the host serial port so that it again matches the power-on reset configuration of the TILE-Gx UART.

To use this command, you need to specify a board type when starting the BTK; otherwise, the normal chip/board probe operations tries to use the UART to access chip registers, which will fail. Then issue the `uart.start_protocol_mode()` command, like this:

```
example$ btk -d target --serial -b wc36
/tmp/tile-btk-qMMzwJ-user/logs created as temporary log directory
tile-btk: Serial device for chip access is /dev/ttyS11
Tilera TILE-Gx Board Test Kit
btk> uart.start_protocol_mode()
```

Forcing UART protocol mode, the console will no longer be functional.

```
UART protocol mode started successfully.
btk>
```

If desired, you can then test the communication path:

```
btk> rshim.test()
Pass 0 progress:
0% |-----| 100%
=====
Finished pass 0
True
btk>
```

You can also just exit the BTK at this point, and then re-enter it without the `-b` flag in order to do the normal chip/board probe operations.

APPENDIX C: BTK TEST LIST

This appendix provides a comprehensive list of BTK tests grouped as follows:

- [Basic Tests](#)
- [Phase 1 Tests](#)
- [Phase 2 Tests](#)
- [Linux Tests](#)

C.1 Additional BTK Tests

[Table C-2](#) provides a list of BTK tests.

C.1.1 Basic Tests

Some Basic tests take optional parameters to their run command. If they do, you can specify them with:

```
tests['testname'].run(param1_name=value, param2_name=value, ...)
```

If any parameter is not specified, the default is used.

Please see [Table C-1](#) for details.

Table C-1. Basic Tests

Test Names	Description	Parameter Name	Description	Default Value
Basic Tests				
chip_verify			Verifies that actual chip SKU and revision match expected values for derived board classes. ^a	

Table C-1. Basic Tests (continued)

Test Names	Description	Parameter Name	Description	Default Value
i2crom	Test all EEPROMs found on I2C by only reading them.	dev_offset	Starting offset.	random
		total_length	Total length of the ROM area to be tested.	rom_size / 32
		op_length	Maximum length of each read operation.	total_length / 32
		pass_count	Number of test passes.	1
i2crom_destructive	Test all EEPROMs found on I2C by writing then reading them. CAUTION: This test may erase important ROM data.	dev_offset	Starting offset.	random
		total_length	Total length of area to be tested.	rom_size / 32
		op_length	Maximum length of each read or write operation.	total_length / 32
		pass_count	Number of test passes.	1
ipic_rshim	For each Inter Processor Interrupt Controller (IPIC) on the chip, test it by sending an IPI to the Rshim.			
jtag_tile_coord_test	Using the JTAG diagnostic path into the tiles, read and verify the tile coordinate register of each tile.			
spd_checksum	Test if valid SPD block is present.			

Table C-1. Basic Tests (continued)

Test Names	Description	Parameter Name	Description	Default Value
srom	Test attached SPI ROM with reads only.	dev_offset	Starting offset.	0
		total_length	Total length of area to be tested.	4096 bytes
		op_length	Maximum length of each read operation.	total_length / 100
		pass_count	Number of test passes.	1
		uncached_read	If True, flush writes to ROM before next read.	True
		reset	If True, reset the SROM at the start of the test.	True
srom_destructive	Test attached SPI ROM with writes and reads. CAUTION: This test is destructive and wipes SROM contents.	dev_offset	Starting offset.	0
		total_length	Total length of area to be tested.	4096 bytes
		op_length	Maximum length of each read or write operation.	total_length / 100
		pass_count	Number of test passes.	1
		uncached_read	If True, flush writes to ROM before next read.	True
		reset	If True, reset the SROM at the start of the test.	True

- a. This board class information specifies chip SKU and memory configurations, which are used during production/manufacturing testing. Parent Board classes generally do not include this test, as they allow for a variety of SKUs (or even experimental chip revisions).

C.1.2 Phase 1, Phase 2, and Linux Tests

Some Phase 1, Phase 2, and Linux tests have parameters associated with them to control test behavior. Unlike Basic tests, these parameters must be changed (if desired) before calling `run()` for the test. Also, the parameter will retain that value set by the user until changed again or the BTK is exited. A parameter is changed by typing at the BTK prompt (`btk>`):

```
tests['testname'].param_name = value
```

The default value for each parameter is the value it has when the BTK is first started.

Please see [Table C-2](#) for details.

Table C-2. Additional BTK Tests—Phase 1, Phase 2 and Linux Tests

Test Names	Description	Parameter Name	Description	Default Value
Phase 1 Tests				
amt	Every tile writes every index in the AMT (Address Mapping Table), reads it back, and checks that the values match. A second pass is done with inverted data. On an error, prints an error indication, failing index, expected data, and actual data.			
bcst	Simple test of the broadcast network. One tile sends out broadcast signals, and all other tiles are expected to receive them.			
bcst_e	Validates the integrity of broadcast signals sent to the east on all four networks across the fabric.			
bcst_n	Validates the integrity of broadcast signals sent to the north on all four networks across the fabric.			
bcst_s	Validates the integrity of broadcast signals sent to the south on all four networks across the fabric.			
bcst_w	Validates the integrity of broadcast signals sent to the west on all four networks across the fabric.			
config_mem	Resets and configures (trains) one memory controller.	primary_mshim:	Selects the memory controller to configure.	0
dtlb	Writes a data pattern to each DTLB entry and reads it back to confirm the integrity of the RAM. The DTLB entries are accessed by index and thus no lookup occurs.			

Table C-2. Additional BTK Tests—Phase 1, Phase 2 and Linux Tests (continued)

Test Names	Description	Parameter Name	Description	Default Value
dtlb_probe	Writes to each DTLB entry and performs a probe to ensure that the new entry is detected. The DTLB entries are accessed by address lookup.			
enet_classifier	Loads the TILE-Gx mPIPE™ classifiers with a simple program that raises a Tile Interrupt and writes the classifier PASS Special Purpose Registers (SPRs). The test checks that the interrupt is asserted in the mPIPE and that the PASS SPR is correct. An error is reported if any check fails. The PASS SPR values are displayed on each pass.	passes	Number of times to run through the test.	50
enet_phy_ext_lpbk	Configures the GbE and XAUI MACs in the TILE-Gx mPIPE for transmit and receive operation. Configures the board level Ethernet PHYs. Triggers the MACs to transmit Ethernet PAUSE packets (100 packets for each XAUI port and 10 packets for each GbE port). With an external Ethernet loopback the PAUSE packet is received by the MAC but not passed to the mPIPE. The test then checks the counters to ensure the proper Tx/Rx statistics and verifies the error counters. An error is reported if any check fails. The sequence is repeated for each speed mode supported by the Ethernet PHY.			
enet_phy_lpbk	This test is the same as the above enet_phy_ext_lpbk, except the packet loopback path is in the on-board Ethernet PHY, if it is supported.			
enet_sd_link	Configures the TILE-Gx mPIPE SerDes for loopback mode, configures the MACs, and verifies that the PCS link status of each SerDes lane. The test is repeated for XAUI with the SerDes running at 3.125 GHz and 6.25 GHz (double rate). GbE is also verified. If the PCS link fails to synchronize and error message is displayed.	double_rate	When True, test at XAUI double rate speed.	True

Table C-2. Additional BTK Tests—Phase 1, Phase 2 and Linux Tests (continued)

Test Names	Description	Parameter Name	Description	Default Value
enet_sd_lpbk	Configures the GbE and XAUI MACs in the TILE-Gx mPIPE for transmit and receive operation. Configures the SerDes for loopback mode. Triggers the MACs to transmit Ethernet PAUSE packets (100 packets for each XAUI port and 10 packets for each GbE port). With the SerDes loopback, the PAUSE packets are received by the MAC but not passed to the mPIPE. The test checks the statistic and error counters, the clock rate and the PCS error status. Any errors are reported. The test is repeated for XAUI with the SerDes running at 3.125 GHz, and 6.25 GHz (double rate). GbE is also verified.	double_rate	When True, test at XAUI double rate speed.	True
enet_serdes_prbs	Configures the SerDes for loopback operation and enables the pseudo random binary sequence (PRBS) generator in the SerDes PMA layer. The PMA's PRBS error status is verified and any errors are reported. This is a XAUI only test and runs with the SerDes configured for 3.125 GHz and then again for 6.25 GHz (double rate).	double_rate	When True, test at XAUI double rate speed.	True
idn_conn	Tests the connectivity of the IDN between each tile and all other tiles.			
ipic	Finds all Inter Processor Interrupt Controller (IPIC), and then for each one, sends an IPI from the rshim to each Tile. Tile IPI_EVENT SPRs are read via JTAG to make sure the IPI arrived. Also for each IPIC, sends all IPI events to one Tile to test the IPIC.			
itlb	Writes a data pattern to each ITLB entry and reads it back to confirm the integrity of the RAM. The ITLB entries are accessed by index and thus no lookup occurs.			
itlb_probe	Writes to each ITLB entry and performs a probe to ensure that the new entry is detected. The ITLB entries are accessed by address lookup.			

Table C-2. Additional BTK Tests—Phase 1, Phase 2 and Linux Tests (continued)

Test Names	Description	Parameter Name	Description	Default Value
l1_dapaths	Walking 1s test for L1D data and address paths to detect stuck at faults and shorts. - On failure, first number indicates address path (0xad1bad1, 0xad1bad2, 0xad1bad3), data path (0xda1bad1), or L1D error status SPR (0xee1bad1) set. - For address or data path failures, prints out the failing address, expected data, and actual data. - For error status failures, prints the status register and the value read for user_params[0].	user_params[0]	Number of iterations to run test.	1
l1_data	Writes a 1 and 0 to every bit in the L1D data array by doing two passes. - On the first pass, for every 8B, the low 4B are CRC'ed and the high 4B are zero. - On the second pass it inverts the bits. - On failure, first number indicates failure on first pass (0x1dbad1), second pass (0x1dbad2), or L1D error status SPR (0x1dbade) set. - For the first or second pass failures, it prints the failing address, expected data, and actual data. - For error status failures, it prints the status register and the value read for user_params[0].	user_params[0]	Number of iterations to run test.	1
l1_data_allcrc	Same as l1_data, except for every 8B, all 8B words are CRC'ed.	user_params[0]	Number of iterations to run test.	1
l1_data_nocrc	Same as l1_data, except for every 8B, alternate all 1's and all 0's.	user_params[0]	Number of iterations to run test.	1
l1_tag	Writes a 1 and 0 to every bit in the L1D tag array by doing two passes with data=address. - On the first pass, write an incrementing value to every tag. - On second pass, invert tag. - When it encounters a failure, the first number indicates the failure on first pass (0x1abad1), on the second pass (0x1abad2), or when the L1D error status SPR (0x1abade) set. - For first or second pass failures, it prints the failing address, expected data, and actual data. - For error status failures, it prints the status register and the values read for user_params[0].	user_params[0]	Number of iterations to run test.	1

Table C-2. Additional BTK Tests—Phase 1, Phase 2 and Linux Tests (continued)

Test Names	Description	Parameter Name	Description	Default Value
l2_dapaths	Walking 1s test for L2 data and address paths to detect stuck at faults and shorts. - When it encounters a failure, the first number indicates the address path (0xad2bad1, 0xad2bad2, 0xad2bad3), data path (0xda2bad1), or L2 error status SPR (0xee2bad1) set. - For address or data path failures, it prints out the failing address, expected data, and actual data. - For error status failures, it prints the status register and the value read for user_params[0].	user_params[0]	Number of iterations to run test.	1
l2_data	Writes a 1 and 0 to every bit in the L2 data array by completing two passes. - On the first pass, for every 8B, the low 4B are CRC'ed and the high 4B are zero. - On the second pass it inverts the bits. - When it encounters a failure, the first number indicates the failure on first pass (0x2dbad1), second pass (0x2dbad2), or L2 error status SPR (0x2dbade) set. - For first or second pass failures, it prints the failing address, expected data, and actual data. - For error status failures, prints the status register and the value read for user_params[0].	user_params[0]	Number of iterations to run test.	1
l2_data_allcrc	Same as l2_data, except for every 8B, all 8B are CRC'ed.	user_params[0]	Number of iterations to run test.	1
l2_data_allcrc_8ways	Same as l2_data_allcrc, except that all 8 ways of the L2 Cache are tested, instead of only 7.	user_params[0]	Number of iterations to run test.	1
l2_data_nocrc	Same as l2_data, except for every 8B, alternate all 1's and all 0's.	user_params[0]	Number of iterations to run test.	1
l2_sharers	This is a two-part test. - In part one, all tiles coherently increment a counter and check that after all tiles are done the counter is set to the correct value. - In part two, each tile drives a subset of connected SDN/RDN wires to both 1 and 0 by performing stores and loads to the home tile. - The home tile is then changed and both parts repeated until every tile has acted as the home tile. No test specific failure messages are sent.	user_params[0]	Number of iterations to run test.	1

Table C-2. Additional BTK Tests—Phase 1, Phase 2 and Linux Tests (continued)

Test Names	Description	Parameter Name	Description	Default Value
l2_structures	Fills the SKF, RTF, MAF, and MDF with local stores and then use loads to check data. When it encounters a failure, the first number indicates the failure during the SKF/RTF portion (0x2d5bad1, 0x2d5bad2) or MAF/MDF portion (0x2d5bad3), followed by the failing address, expected data, and actual data.	user_params[0]	Number of iterations to run test.	1
l2_tag	Writes a 1 and 0 to every bit in the L2 tag array by doing two passes with data=address. - On the first pass, write an incrementing value to every tag. - On second pass, invert tag. - On failure, first number indicates failure on first pass (0x2abad1), second pass (0x2abad2), or L2 error status SPR (0x2abade) set. - For first or second pass failures, prints the failing address, expected data, and actual data. - For error status failures, prints the status register and the value read for user_params[0].	user_params[0]	Number of iterations to run test.	1
pcie_serdes_config	Test Writing and Reading PCIe Serdes Config Registers. The test automatically runs on all available PCIe SerDes of the Gx chip under test.			
regfile01	Writes data patterns into each general purpose register (GPR) and reads them back to confirm the integrity of the register file.			
regfile2	Writes each GPR with the result of a load that hits in the data cache. This test covers a dedicated write port to the register file.			
regfile3	Writes each GPR with the result of a load that misses in the data cache. This test covers a dedicated write port to the register file.			
regfile_bypass	Tests the GPR bypass paths from each pipeline slot to every other pipeline slot. "X" bundles (two-wide) and "Y" bundles (three-wide) are both covered.			
regfile_read	Covers read ports of the general purpose register file without sensitizing the speed paths as regfile_read_2 and regfile_read_3 do.			

Table C-2. Additional BTK Tests—Phase 1, Phase 2 and Linux Tests (continued)

Test Names	Description	Parameter Name	Description	Default Value
regfile_read_2	Covers four of seven read ports of the general purpose register file while sensitizing the path where the value is written and read in the same cycle.			
regfile_read_3	Covers two of the seven read ports of the general purpose register file while sensitizing the path where the value is written and read in the same cycle. Note the seventh port is only used for the address of a store and is covered by the cache tests.			
regfile_write	Writes and reads all bits of general purpose registers. This test also covers the three write ports of the register file logic.			
shim_AMT_test	Finds all the IO shim AMTs, and then writes and reads several data patterns to test that all bits are correct.			
simple_dll_test	Performs a reset of the DLL of the selected memory interface (mshim).	primary_mshim	Selected mshim.	0
udn_conn	Tests the connectivity of the UDN between each tile and all other tiles.			
xdn_demux	Causes the XDN demux buffer to be filled with payload words of specific data patterns. The incoming packets are then dequeued and compared against the original payloads. Each tile sends UDN packets to one other tile such that all tiles serve as a sender and receiver.			
Phase 2 Tests				
bcst_rsh	The rshim sends out broadcast signals on all four networks, and all tiles expect to receive those signals. One tile sends out broadcast signals on all four networks, and the rshim is expected to receive them.			
bringup_memory	Basic test of one memory controller.	primary_mshim	Selects the memory controller.	0
		iterations	Controls number of iterations of the core test loop each tile runs.	10,000

Table C-2. Additional BTK Tests—Phase 1, Phase 2 and Linux Tests (continued)

Test Names	Description	Parameter Name	Description	Default Value
compression	Compresses a buffer containing specific data patterns using the hardware DEFLATE engine, then decompresses that encoded data using the hardware INFLATE engine. The final data is compared against the original.			
compress_text	Compresses the text of Shakespeare's "As You Like It" (a component of the standard Canterbury Corpus) using the hardware DEFLATE engine, then decompresses it using both instances of the INFLATE engine within a MiCA™ block. This test provides better coverage of the compression hardware (INFLATE and DEFLATE) than does the test compression.	user_params[0]	Compression instance to use (0 or 1).	0
crypto_aes	Four tiles each perform one AES encrypt/decrypt operation and three encrypt-only operations. This test is repeated X times on the specified cryptographic instance. When it encounters a failure, the test prints the total number of incorrect 64-bit values.	user_params[1]	Number of iterations (default = 1; specifying 0 will do 1 iteration).	1
		user_params[2]	Crypto instance to use (0 or 1).	0
crypto_fro_counts	Test TRNG free-running-oscillators (FRO). Prints the value of counts for each FRO. The actual values vary based on chip frequency, temperature, and voltage, as well as individual variance of the FROs. The count values for the FROs should be monotonically increasing.	user_params[1]	Selects on which Tile the test is run.	0
crypto_ops	All tiles used to test a combination of Kasumi, SNOW, and Packet cryptographic operations and checks expected data. This test is repeated X times on the specified cryptographic instance. When it encounters a failure, it test prints total number of failing operations.	user_params[1]	Number of iterations (default = 1; specifying 0 will do 1 iteration).	1
		user_params[2]	Crypto instance to use (0 or 1).	0
crypto_pka	This cryptographic public key accelerator test tests one operation in flight per ring of PKA. It then prints some information about the operation, and if it encounters a failure it prints expected and actual data.	user_params[1]	Controls the number of iterations.	1
crypto_pka_2	This test is the same as crypto_pka test, but with multiple operations in flight per ring.	user_params[1]	Controls the number of iterations.	1

Table C-2. Additional BTK Tests—Phase 1, Phase 2 and Linux Tests (continued)

Test Names	Description	Parameter Name	Description	Default Value
crypto_trng	TRNG test. Collects random numbers to test basic functionality of the TRNG. Prints some status information.			
crypto_trng_ipi	TRNG interrupt test. Collects random numbers to test the alarm interrupt function. The test prints some status information related to TRNG.			
crypto_zip	Using MICA, one tile will first encrypt a random packet, followed by another tile compressing and decompressing the packet, and finally the original tile will decrypt the packet and check the result. This is repeated iteration times on the specified crypto and compression instance.	user_params[0]	Verbosity level (0, 1, or 2; higher number prints more messages).	0
		user_params[1]:	Number of iterations (default = 1; specifying 0 will do 1 iteration).	1
		user_params[2]	Crypto and compression instance to use (0 or 1).	0
data_mover	Runs a single memory copy operation on each cryptographic and compression instance for a total of four operations performed and checks copied data. When it encounters a failure, the test prints the total number of context errors plus memory miscompares.			
mem_stress	Stress test of one memory controller, emphasizing the intensity and rate of memory traffic.	primary_mshim	Selects the memory controller.	0
		iterations	Controls number of iterations of the core test loop each tile runs.	100,000
mem_stress_mshim0	Same as mem_stress, but always tests interface 0.			
mem_stress_mshim1	Same as mem_stress, but always tests interface 1.			
mem_stress_mshim2 ^a				
mem_stress_mshim3 ^a				

Table C-2. Additional BTK Tests—Phase 1, Phase 2 and Linux Tests (continued)

Test Names	Description	Parameter Name	Description	Default Value
mpipe_gbe_mac_lpbk ^{b, c}	Utilizes the mPIPE Ethernet loopback application and enables all 16 GbE MACs with the loopback functionality provided in the GbE MACs. This test does not exercise the PCS or SerDes but is a good indication that the mPIPE, GbE MACs, and DDR memory are functional. This test sends 0x10000 packets.			
mpipe_gbe_sd_lpbk ^{a, b}	Utilizes the mPIPE Ethernet loopback application and enables all 16 GbE MACs and the loopback functionality provided in the SerDes. This test sends 0x10000 packets.			
mpipe_lb_chan_lpbk ^{a, b}	Utilizes the mPIPE Ethernet loopback application and enables all four mPIPE loopback channels. No MACs are involved in this test. This test sends 0x40000 packets.			
mpipe_phy_ext_lpbk ^{a, b}	Utilizes the mPIPE Ethernet loopback application. The Python wrapper discovers the PHYs and, based on their defined functionality (enet_phy.py), configures the MACs accordingly. This test includes the number/type of MACs and speed/mode. The test assumes the user has connected the appropriate external loopback, an SFP+ with a fiber loopback, for example. If the phy is XAUI, 0x2000 packets will be sent. If the phy is GbE, then 0x1000 packets are sent.			
mpipe_phy_lpbk ^{a, b}	Utilizes the mPIPE Ethernet loopback application and enables loopback functionality in the on board Ethernet PHYs. The Python wrapper discovers the PHYs and based on their defined functionality (enet_phy.py) will configure the MACs accordingly. This includes the number/type of MACs and speed/mode. If the phy is XAUI, 0x2000 packets will be sent. If the phy is GbE, then 0x1000 packets will be sent.			
mpipe_xaui_sd_lpbk ^{a, b}	Utilizes the mPIPE Ethernet loopback application and enables all four XAUI MACs and the loopback functionality provided in the SerDes. This test sends 0x40000 packets.			
shared_mem	Each tile performs a combination of various memory operations, including synchronized increments of multiple shared counters, designed to stress the cache hierarchy. The test utilizes pseudo random number SPRs.			
trio_loopback ^d	Sets up a PCIe port as StreamIO and the SerDes in internal loopback. Then, using the TRIO engine, moves data across the ports.	mac_num	Specifies which StreamIO MAC to use.	0

Table C-2. Additional BTK Tests—Phase 1, Phase 2 and Linux Tests (continued)

Test Names	Description	Parameter Name	Description	Default Value
uncacheable	Each tile performs a series of independent uncacheable stores, then all tiles read back all addresses and checks the value. On failure, test prints encoded number where each of the bits [7:0], [15:8], [23:16], [31:24] is a separate failure count of how many tiles read an incorrect value.			
Linux Tests				
linux_memtest	Verify DDR RAM with linux diagnostics.	core_freq	Core Frequency in MHz.	Default is max allowed for board.
		ddr_freq	DDR speed in Mega Transfers (MTs) per second).	Default is max for board and/or DIMMs.
		passes		10

- a. These are TILE-Gx72 only tests.
- b. This class of tests utilizes a common mPIPE Ethernet loopback application. A master Tile is responsible for mPIPE setup/tear-down and memory initialization. Worker Tiles are responsible for packet generation and checking. Packets are generated and egressed by Tile code for each channel/MAC-enabled. The application assumes that packets are looped back, Tx to Rx, at the Ethernet interface, or by the dedicated loopback channels. Packet sizes are varied and all ingress packets are compared against the egress packets by the worker Tile code.

This application is designed to support the entire TILE-Gx family of processors. It utilizes multiple Tiles with each handling multiple “flows” through the use of interrupts. A flow is defined as an eDMA/Notif ring pair. The classifier is programmed to dispatch packets to a flow, based on the channel number. This is a generic loopback application capable of utilizing any XAUI or GbE MAC and multiple forms of loopback. Because of the generic nature of this test, and multiple flows per Tile, this test does not reach line rate. Python test wrappers provide test parameters, via the boot process, to create the various test scenarios. The parameters are numerous, refer to the EnetTest class in `enet_test.py`. The wrappers typically configure a default packet count and number of MACs to test, which can be overridden by the user. Messages are displayed to indicate a test’s progress. Errors from the test display self-explanatory messages, with the exception of a test time out. A test time out typically occurs when the mPIPE is waiting for an ingress packet and never receives one. This can be attributed to a number of causes; improper configuration, memory issues, or loopback setup issues.

- c. For chips with two mPIPE shims (such as TILE-Gx72) replace “mpipe” with “mpipe0” or “mpipe1” in the `testname` to run the test on shim 0 or 1, respectively.
- d. For chips with two trio shims (such as TILE-Gx72), there are two tests, `trio0_loopback` and `trio1_loopback`, which run the test on shim 0 or 1, respectively.

APPENDIX D: TEST QUALIFIERS

This appendix provides a preliminary description of qualifiers.

D.1 Selecting Tests

Qualifiers are a set of characteristics that describe a test. A test can have many qualifiers. For example, a test can be MPIPE and a LOOPBACK test.

The command `board.set_qual(qualifiers=(), disqualifiers=(), qual_ops='or')` is used in conjunction with `board.show_tests()` and `board.run_tests()`. This command takes as arguments *qualifiers*, *disqualifiers*, and an *operation type*.

The *qualifiers* parameter lists the attributes or set of qualifiers a test can have for it to be selected.

The *disqualifiers* parameter lists the attributes or set of qualifiers that disqualify a test from being selected.

The *qual_ops* parameter determines how the other two parameters are used to select test. The default is Logical OR behavior. If any qualifier in the *qualifiers* list matches a test qualifier, then that test is selected. Then, anything in the *disqualifiers* list that matches an already selected test will remove it as a selected test.

If *qual_ops*='and' is used for Logical AND behavior, all attributes selected in *qualifiers* or *disqualifiers* must be in a test's qualifier list for that test to be selected or deselected.

D.2 Running Qualified Tests

The default behavior of `board.show_tests()` is to show all tests. Once `board.set_qual()` is used to select qualifiers, all Basic, Phase1, Phase2, and Linux tests shown with `show_tests` are qualified tests. These are the tests that execute with the `board.run_tests()` command. In this manner, tests with attributes in common can be selected to run as a test group.

For example, to run all tests that are Ethernet Loopback tests that do not require External loopback connections, you could do:

```
board.set_qual(qualifiers=('enet', 'loopback'), disqualifiers=('ext'),
qual_ops='and')
board.run_tests()
```

To run both ROM and Ethernet tests, but only those that are neither manual tests nor destructive:

```
board.set_qual(qualifiers=('enet', 'rom'), disqualifiers=('manual',
'destructive'), qual_ops='or')
board.run_tests()
```

For a complete list of qualifiers, refer to Table D-1, "Test Qualifiers," on page 74.

D.3 Selecting Test Phases

All regression scripts and `board.run_tests()` have an optional parameter that includes tests to be run based on their category (Basic, Phase1, Phase2, and Linux). This parameter is a list of categories set like this: `board.run_tests(phase=[0, 1, 2, 3])`. The default behavior is to select all four categories (Basic=0, Phase1=1, Phase2=2, and Linux=3).

It is often useful to select tests based on these categories as it allows for test selection based on board/host conditions or restrictions.

D.4 Current List of Test Qualifiers

[Table D-1](#) provides a list of qualifiers.

Table D-1. Test Qualifiers

Qualifiers	Description
internal	Test is for Tileria internal use.
board	Board level tests.
example	Example tests.
development	In development.
characterization	Chip characterization may not have PASS/FAIL.
loopback	Loopback test.
memory	Memory targeted test.
chip	Test core chip functions, does not test functions that rely on external devices such as ROMs.
mpipe	Mpipe functionality.
rom	Tests ROM devices.
gpio	GPIO pin tests.
enet	Ethernet test.
gbe	Test Ethernet for GbE interfaces.
xaui	Test Ethernet for XGBE/XAUI interfaces.
mac	Target MAC functionality.
serdes	Target SERDES functionality.
phy	Target PHY functionality.

Table D-1. Test Qualifiers (continued)

Qualifiers	Description
ext	External connections required.
manual	User interaction required; do not run autonomously.
custom	Test should not be run as is requires customization.
destructive	Original contents/state is not restored.
linux	TileGx is booted with linux and test is run under that.
production	Targeted for board manufacturing process.
Network	Qualify Linux tests that require Ethernet.
BIB	For testing and for writing BIBs or AIBs
BMC	Test specifically targets the BMC.
IPMI	Uses IPMI commands
multi	Multiple Linux tests executed with a single command.
margin	Group of tests for Frequency or Voltage margining.
UART	Test UART interfaces.
PCIE	Test PCIe interfaces.

APPENDIX E: GLOSSARY, CONVENTIONS AND STANDARDS

This appendix provides a glossary of terminology commonly used in describing the Board Test Kit, board configurations and hardware, and BTK software.

E.1 Glossary

Term	Definition
AIB	Additional Information Block. Since the BIB is not intended to be modified, an AIB is created for components that are changeable, like removable DIMMs or mezza-nine cards. An AIB has many of the same parts, referred to as item types, as a BIB. It is added to an I²C EEPROM on the optional module. Each BIB on the main board contains an AIB item that points to the additional EEPROM. If that EEPROM is present at boot time, entries from the AIB in that EEPROM are effectively incorporated, by reference, into the BIB. The BIB is copied into memory at boot time, and then items from any AIBs are inserted into that in-memory copy, which is then used to satisfy any requests for BIB information; the BIB is not modified.
Auto-discovery	The process of discovering the properties of a TILE-Gx and board or system type by querying registers through the RSHIM interface at BTK startup. When the BTK uses Auto-discovery on startup, it also queries SROM or I2CROM devices to load the BIB. An additional process identifies the correct board class to use in the BTK session, based on part numbers found in BIB and optional AIB entries. This document occasionally uses the term, Auto-discovery, to include all steps involved in this process.
BIB	Board Information Block. The BIB is designed to describe characteristics of a board or system that are expected to be stable. For example, a board's serial number or an Ethernet interface's MAC address do not normally change after the board is manufactured. The BIB is copied into memory at boot time. Each BIB contains a header, a set of items (zero or more item descriptors), and a trailer.
BMC	Baseboard Management Controller. The BMC provides for chassis management and CPU module management. The BMC processor is the Aspeed AST2300. External communication to the processor can be via USB, RS-232, or Fast (10/100) Ethernet. CPU module communication with the BMC can be via I ² C, USB, or PCIe.
board	A collection of GX Nodes and associated devices.
board devices	A dictionary of board device interfaces accessible from the board interface.
board management	References BMC interactions with a Tilera Processor or its host board.
chip	Tilera's TILE Processor.
chip access	Interface from the host to a Tilera® Processor on a board (typically a PCIe, USB, or UART).

Term	Definition
IPIC	Inter-Processor Interrupt Controller that monitors behavior of Inter-Processor Interrupt (IPIs). An IPI is a special type of interrupt by which one processor can interrupt another processor in a multiprocessor system if the interrupting processor requires action from the other processor.
IPMI	Intelligent Platform Management Interface is used for out-of-band monitoring and management of thermal, power, and other attributes of a system.
label	Identifier for an interface specified in a configuration file; used by <code>--dev</code> as an alias for that interface.
manage	An optional BMC interface accessible through the BTK to monitor or control Tilera Processors.
module	One of several interchangeable units within a platform.
node	A TILE-Gx chip and all devices interfaced through it. Note that the term <i>node</i> as used in this manual is interchangeable with CPU _n A or CPU _n B (where multiple CPUs are present).
platform	A collection of one or more interchangeable units that collectively form a Tilera hardware solution.

I INDEX

A

accessing information on an interface
via help() [16](#)

adding

a new my_new_board.py file [38](#)

any new tests specific to the board [38](#)

new PHYs [40](#)

new SROMs [40](#)

subclasses to your new board file [38](#)

additional BTK tests [59](#)

basic tests

chip_verify [59](#)

i2crom [60](#)

i2crom_destructive [60](#)

ipic_rshim [60](#)

jtag_tile_coord_test [60](#)

spd_checksum [60](#)

srom [61](#)

srom_destructive [61](#)

Linux tests

linux_memtest [72](#)

phase 1 tests

amt [62](#)

bcst [62](#)

bcst_e [62](#)

bcst_n [62](#)

bcst_s [62](#)

bcst_w [62](#)

config_mem [62](#)

dtlb [62](#)

dtlb_probe [63](#)

enet_classifier [63](#)

enet_phy_ext_lpbk [63](#)

enet_phy_lpbk [63](#)

enet_sd_link [63](#)

enet_sd_lpbk [64](#)

enet_serdes_prbs [64](#)

idn_conn [64](#)

ipic [64](#)

itlb [64](#)

itlb_probe [64](#)

l1_dapaths [65](#)

l1_data [65](#)

l1_data_allcrc [65](#)

l1_data_nocrc [65](#)

l1_tag [65](#)

phase 1 tests (*continued*)

l2_dapaths [66](#)

l2_data [66](#)

l2_data_allcrc [66](#)

l2_data_nocrc [66](#)

l2_sharers [66](#)

l2_structures [67](#)

l2_tag [67](#)

pcie_serdes_config [67](#)

regfile_bypass [67](#)

regfile_read [67](#)

regfile_read_2 [68](#)

regfile_read_3 [68](#)

regfile_write [68](#)

regfile01 [67](#)

regfile2 [67](#)

regfile3 [67](#)

shim_AMT_test [68](#)

simple_dll_test [68](#)

udn_conn [68](#)

xdn_demux [68](#)

phase 2 tests

bcst_rsh [68](#)

bringup_memory [68](#)

compress_text [69](#)

compression [69](#)

crypto_aes [69](#)

crypto_fro_counts [69](#)

crypto_ops [69](#)

crypto_pka [69](#)

crypto_pka_2 [69](#)

crypto_trng [70](#)

crypto_trng_ipi [70](#)

crypto_zip [70](#)

data_mover [70](#)

mem_stress [70](#)

mem_stress_mshim0 [70](#)

mem_stress_mshim1 [70](#)

mpipe_gbe_mac_lpbk [71](#)

mpipe_gbe_sd_lpbk [71](#)

mpipe_lb_chan_lpbk [71](#)

mpipe_phy_ext_lpbk [71](#)

mpipe_phy_lpbk [71](#)

mpipe_xaui_sd_lpbk [71](#)

shared_mem [71](#)

trio_loopback [71](#)

- phase 2 tests (*continued*)
 - uncacheable [72](#)
- address hashing [48](#)
- Address Mapping Table
 - See AMT [16](#)
- advanced custom configurations [42](#)
- AIB
 - defined [77](#)
- AMT [16, 68](#)
- amt [62](#)
- associated PCIe port
 - training [21](#)
- auto-discovery [35, 39](#)
 - defined [77](#)
 - failures [42](#)
- B**
- b flag [58](#)
- basic aliveness test
 - runs [15](#)
- basic tests [59](#)
- bcst [62](#)
- bcst_e [62](#)
- bcst_n [62](#)
- bcst_rsh [68](#)
- bcst_s [62](#)
- bcst_w [62](#)
- BIB [7, 37, 43](#)
 - defined [77](#)
 - defining [38](#)
 - entry [42](#)
 - reading the BIB [39](#)
 - writing the BIB to a SPI ROM [39](#)
- BIBs [37](#)
- BMC
 - console
 - defining [10](#)
 - defined [77](#)
- bmc {HOSTNAME} [13](#)
- board [12, 39](#)
- board
 - class
 - registering [39](#)
 - registering using auto-discovery method [39](#)
 - classes [59](#)
 - defined [77](#)
 - devices [29](#)
 - defined [77](#)
 - file
 - creating new files [38](#)
 - management
 - defined [77](#)
 - tests
 - adding new tests [38](#)
- Board Information Block
 - See BIB
- board parameter [12](#)

- Board Test Kit
 - See BTK
- board_devices [32, 33](#)
- board_regress [32](#)
- board.bib.dump() [39](#)
- board.board_devices [29](#)
- board.chip_access_points [15](#)
- board.clear_all_test_status() [16](#)
- board.get_core_voltage() [15](#)
- board.get_cpu_temperature() [15](#)
- board.get_ddr_voltage() [15](#)
- board.i2c_discovery() [15](#)
- board.print_all_test_status() [16](#)
- board.set_clock_voltage_pairs() [15](#)
- board.set_core_voltage() [15](#)
- board.set_ddr_voltage() [15](#)
- board.show_tests() [16, 31](#)
- board.switch_chip_access(access) [15](#)
- board/BIB creation
 - flowchart [37](#)
- boot_on_console [55](#)
- boot.set_mshim(interface_num) [47](#)
- booting
 - using dual-UART methods [51](#)
 - using single-UART methods [51](#)
- bringing up
 - the Ethernet PHY [41](#)
 - the user interface [9](#)
- bringup_memory [68](#)
- BTK [1](#)
 - contents [1](#)
 - locating [1](#)
 - logs locations [34](#)
 - options [9](#)
 - overview [1](#)
 - purpose [1](#)
 - tests
 - additional [59](#)
 - running [31](#)
 - users [1](#)
 - using after booting [57](#)
- btk prompt [9, 28](#)
- BTK test list [59](#)
- btk.log [33, 34](#)
- Buffer Stack Manager setup and buffers
 - ensuring legality [23](#)

- C**
- calculating the home tile for a given physical address
 - assuming hash-for home memory [15](#)
- changing chip access to another device [15](#)
- channel number [72](#)
- checking
 - EDMA setup state [23](#)
 - IDMA setup state [23](#)
 - network and print picture of network blockages [16](#)
 - PCS status [24](#)

- chip
 - access
 - defined 77
 - auto-discovery 12
 - defined 77
 - I/O and clock rates
 - printing 16
 - SKU 59
 - chip help 12
 - chip parameter 12
 - chip_access mechanism 38
 - chip_verify 59
 - chip.calc_home(pa, offset=0, mask=15) 15
 - chip.alive() 7, 15, 32, 33
 - chip.boot_uart() 54
 - chip.cmem_read() 16
 - chip.core_clock_rate() 15
 - chip.dump_f2v() 16
 - chip.measure_refclk(measure_seconds=4.0) 16
 - chip.perf_rates() 16
 - chip.print_network_state(network) 16
 - chip.print_pcs_and_lrs() 16
 - chip.print_pcs() 16
 - chip.reset() 15, 47
 - chip.rev() 16
 - chip.set_core_clock_rate(rate) 15
 - chip.show() 16
 - clearing
 - any error status registers in the AdvancedErrorReporting
 - structure 22
 - cumulative test status 16
 - GPIO (low) 29
 - commands
 - common to all units 16
 - for MACs in GbE mode 24
 - for MACs in XAUI mode 24
 - for TILES 18
 - issuing 15
 - comp0 17
 - compress 51
 - compress_text 69
 - compression 69
 - config_mem 62
 - config.cclk_freq = freq 47
 - configuration files 10
 - configuring
 - a memory interface and running a test 43
 - a port for PCIe
 - operations 22
 - a port for StreamIO operations 22
 - the BTK to work with a custom board 38
 - the MAC 24
 - connect_to_chip() 38
 - contents vii
 - control the DDR3 operating voltage 44
 - core clock
 - rate
 - core clock (*continued*)
 - setting 15
 - core voltage
 - reading the setting of 15
 - returning the setting of 15
 - specifying 15
 - counters from the mPIPE unit
 - dumping 23
 - cpu number 18
 - create 51
 - create_access_points() 38
 - create_bib function 39
 - creating
 - a new board file 38
 - multiple custom board classes 42
 - objects that group i2c devices together 27
 - crypto_aes 69
 - crypto_fro_counts 69
 - crypto_ops 69
 - crypto_pka 69
 - crypto_pka_2 69
 - crypto_trng 70
 - crypto_trng_ipi 70
 - crypto_zip 70
 - crypto0 17
 - current
 - clock rate
 - returning 15
 - PC and LR of all tiles
 - printing 16
 - port state
 - providing 21
 - custom 12, 14
 - custom
 - board classes
 - creating multiples 42
 - boards
 - configuring the BTK to work with a custom board 38
 - configurations
 - advanced 42
 - custom board
 - configuring 38
 - custom board initialization 38
 - custom.py 38
 - customer support ix
 - customizing
 - BIB entries 38
 - board reset instructions 38
 - custom-options 14
- ## D
- data_mover 70
 - DDR3
 - memory 8
 - voltage
 - returning 15
 - specifying 15

- debug.check_all_tiles_L2 (physical_address) [48](#)
- debug.print_network_state () [48](#)
- debug.print_pcs_and_lrs() [48](#)
- debug.print_pcs() [48](#)
- debug.print_tile_state (x_coordinate, y_coordinate) [48](#)
- debug.read_spr(x_coordinate, y_coordinate, SPR index) [49](#)
- debugging command [18](#)
- def __add_tests [38](#)
- def __init_board [38](#)
- def create_bib [38](#)
- def reset [38](#)
- defining
 - I2cAccess classes [27](#)
 - reset instructions [38](#)
 - the BIB [38](#)
 - the BMC console [10](#)
 - the device mappings for a bus [27](#)
- determining PA and homing information for a home Tile [18](#)
- dev {NAME|LABEL} [9](#)
- dev command [78](#)
 - LABEL option [10](#)
 - NAME option [10](#)
 - none option [10](#)
- device
 - classes [26](#)
 - ID
 - finding matching devices(s) [23](#)
 - option [7](#)
- DIMM sockets [43](#)
- dir() [15](#), [16](#), [33](#)
- disabling
 - MAC Tx to Rx loopback [24](#)
 - one port [22](#)
 - the configuration/use of error correction (ECC) [44](#)
- discovering I2C devices on i2cm busses [15](#)
- display_msh_config() [48](#)
- display_spd() [48](#)
- displaying
 - all cumulative test errors [16](#)
 - the frequency-to-voltage table [16](#)
 - the SPD information [44](#)
 - TILE-Gx temperature sensors [15](#)
 - valid chip access interfaces [15](#)
- displays
 - suppressing output displays [35](#)
- disqualifiers parameter [73](#)
- documents
 - related [viii](#)
- dtlb [62](#)
- DTLB contents
 - dumping [20](#)
- dtlb_probe [63](#)
- dual-UART booting [51](#)
- dumping
 - enabled DMA rings for pull [21](#)
 - enabled DMA rings for push DMA [21](#)
 - information for all enabled eDMA rings [23](#)
- dumping (*continued*)
 - the contents of the DTLB [20](#)
 - the contents of the Home Mapping Table [20](#)
 - the contents of the ITLB [20](#)
 - the contents of the L1 Dcache [20](#)
 - the contents of the L1 Icache [20](#)
 - the subset of the tile's state [18](#)
 - various counters from the mPIPE unit [23](#)
- E**
- ECC [44](#)
- eDMA
 - ring
 - printing related information [23](#)
 - setup
 - checking the state [23](#)
- enabling
 - DMA rings for pull DMA dumping [21](#)
 - DMA rings for push DMA dumping [21](#)
- eDMA rings
 - dumping [23](#)
 - MAC Tx to Rx loopback [24](#)
 - SerDes/PCS loopback, MAC Rx/Tx [24](#)
- enet_classifier [63](#)
- enet_phy_ext_lpbk [63](#)
- enet_phy_lpbk [63](#)
- enet_sd_link [63](#)
- enet_sd_lpbk [64](#)
- enet_serdes_prbs [64](#)
- ensuring legality of Buffer Stack Manager setup and buffers [23](#)
- enumerating the PCIe subsystem [22](#)
- error status registers in the AdvancedErrorReporting structure
 - clearing [22](#)
- example log directory [34](#)
- execfile() [9](#), [14](#)
- executing
 - Python instructions [9](#)
- F**
- filename [54](#)
- finding device(s) matching vendor ID and/or device ID [23](#)
- flowchart
 - creating our own board/BIB [37](#)
- G**
- GbE packet counters
 - verifying [24](#)
- general commands [15](#)
- general purpose registers
 - See* GPRs
- GPIO
 - BTK commands [29](#)
 - clearing (low) [29](#)
 - pins
 - setting the state [29](#)

GPIO (*continued*)
 reading [29](#)
 setting (high) [29](#)
 gpio [17](#)
 GPRs [18](#), [19](#), [67](#)
 grid format
 printing [16](#)
 gx_regress script [32](#)

H

hash [55](#)
 hash-for-home memory [15](#)
 help() [16](#), [49](#)
 marker [16](#)
 help(interface) [15](#)
 help(unit_name) [18](#)
 Home Mapping Table
 dumping the contents [20](#)
 host
 requirements [3](#)

I

i2crom [60](#)
 i2crom_destructive [60](#)
 identifying
 PHYs in MDIO space [24](#)
 revision ID of TILE-Gx processor [16](#)
 IDMA setup
 checking the state [23](#)
 idn_conn [64](#)
 import
 command [14](#)
 my_new_board [38](#)
 initializing the custom board [38](#)
 installing
 drivers [3](#)
 the kit [4](#)
 Inter Processor Interrupt Controller
 See IPIC
 interfaces
 listing all accessible interfaces [16](#)
 IPIC [60](#), [64](#)
 defined [78](#)
 ipic [64](#)
 ipic_rshim [60](#)
 IPMI
 defined [78](#)
 issuing
 commands [15](#)
 soft reset to TILE-Gx processor [15](#)
 ITLB
 dumping contents [20](#)
 itlb [64](#)
 itlb_probe [64](#)

J

jtag_tile_coord_test [60](#)

K

kit
 installing [4](#)

L

L1 Dcache
 dumping [20](#)
 L1 Icache
 dumping the contents [20](#)
 l1_dapaths [65](#)
 l1_data [65](#)
 l1_data_allcrc [65](#)
 l1_data_nocrc [65](#)
 l1_tag [65](#)
 l2_dapaths [66](#)
 l2_data [66](#)
 l2_data_allcrc [66](#)
 l2_data_nocrc [66](#)
 l2_sharers [66](#)
 l2_structures [67](#)
 l2_tag [67](#)
 label [78](#)
 defined [78](#)
 LABEL option [10](#)
 link errors
 testing for [23](#)
 Linux cpu number [18](#)
 Linux tests [31](#)
 linux_memtest [72](#)
 listing
 all tests available [16](#)
 all the interfaces that you can use [16](#)
 available routines with descriptions [18](#)
 locating the kit [1](#)
 location of BTK logs [34](#)
 --log [13](#), [33](#)
 log directory [33](#)
 logging [33](#)
 logging and print levels
 critical [33](#)
 debug [33](#)
 error [33](#)
 info [33](#)
 setting [33](#)
 warning [33](#)
 logging levels [33](#)
 critical [33](#)
 debug [33](#)
 error [33](#)
 info [33](#)
 warning [33](#)
 logging module
 Python [33](#)
 Logical AND behavior [73](#)
 Logical OR behavior [73](#)
 logs
 test [34](#)

M

- m
 - MAC or port number in TRIO RC commands [22](#)
- MAC
 - commands [24](#)
 - configuring [24](#)
 - number associated with this channel
 - returning [23](#)
 - Rx/Tx
 - enabling [24](#)
 - Tx to Rx loopback
 - enabling or disabling [24](#)
- main BTK log [33](#)
- manage [11](#)
- manage
 - defined [78](#)
- manual contents [vii](#)
- MDIO PHY
 - register
 - reading using clause 22 [24](#)
 - reading using clause 45 [24](#)
 - writing using clause 22 [24](#)
 - writing using clause 45 [24](#)
- MDIO space
 - identifying PHYs [24](#)
- measuring the refclk rate in MHz [16](#)
- mem_dump(address, length) [45](#)
- mem_read(address, size) [45](#)
- mem_stress [70](#)
- mem_stress_mshim0 [70](#)
- mem_stress_mshim1 [70](#)
- mem_write(address, data, size) [45](#)
- memory
 - from home tile
 - reading [16](#)
 - interface
 - configuring [43](#)
 - shim (msh) object [44](#)
- module
 - defined [78](#)
- mPIPE
 - commands [23](#)
 - Ethernet loopback application [72](#)
- mpipe_gbe_mac_lpbk [71](#)
- mpipe_gbe_sd_lpbk [71](#)
- mpipe_lb_chan_lpbk [71](#)
- mpipe_phy_ext_lpbk [71](#)
- mpipe_phy_lpbk [71](#)
- mpipe_xaui_sd_lpbk [71](#)
- msh object [44](#)
- mshim object [45](#)
- my_new_board.py file
 - adding a new file [38](#)

N

- NAME option [10](#)

- network blockages
 - checking for [16](#)
- no-completion [13](#)
- node [13](#)
- node
 - defined [78](#)
- none option [10](#)
- notation conventions [ix](#)

O

- obtaining
 - product information [ix](#)
- one DMA descriptor
 - printing [21](#)
- other Tile members [20](#)
- overview of the BTK [1](#)

P

- PA [18](#)
- PA and homing information
 - determining for a home Tile [18](#)
- parts2class method [6, 38](#)
- PCIe
 - commands [20](#)
 - operations
 - configuring a port [22](#)
 - subsystem
 - enumerating [22](#)
 - tree
 - printing information about the previous bus crawl [22](#)
- pcie_serdes_config [67](#)
- PCS status
 - checking [24](#)
- performing a quick test of all the devices [23](#)
- phase 1 tests [62](#)
- phase 2 tests [68](#)
- phases parameter [32](#)
- PHY
 - adding new [40](#)
- Physical Address
 - See* PA
- physical address
 - calculates home tile's address [15](#)
- platform
 - defined [78](#)
- pmgr [29](#)
- pmon [29](#)
- port [54](#)
- ports
 - printing summary information [23](#)
- Power Managers
 - See* pmgr
- Power Monitors
 - See* pmon
- PowerMonitor class [28](#)
- primary_mshim attribute [44](#)

- printing
 - graphic showing chip I/O and clock rates [16](#)
 - information related to one eDMA ring [23](#)
 - information related to push_dma ring [21](#)
 - one DMA descriptor, given ring number and position in the ring [21](#)
 - one eDMA descriptor
 - eDMA descriptor
 - printing [23](#)
 - summary of information for each port [23](#)
 - summary of state of all the MACs on this trio shim [21](#)
 - the current PC and LR of all tiles in a grid format [16](#)
 - the current PC of all tiles in a grid format [16](#)
 - the PCIe tree based on a previous bus crawl [22](#)
- product information [ix](#)
- prompt
 - bt> [9](#)
- providing the current port state [21](#)
- push_dma ring
 - printing information [21](#)
- Python
 - code
 - using custom [14](#)
 - command-line interpreter [9](#)
 - instructions
 - executed [9](#)
 - only tests [31](#)
 - test wrappers [72](#)
- Q**
 - qual_ops parameter [73](#)
 - qualifiers parameter [73](#)
 - quiescing the tile [18](#)
- R**
 - rarely used options [13](#)
 - RDLAT [8](#)
 - read function [25](#)
 - read latency
 - See RDLAT
 - reading
 - an MDIO PHY register using clause 22 [24](#)
 - an MDIO PHY register using clause 45 [24](#)
 - GPIO [29](#)
 - memory from home tile [16](#)
 - the BIB [39](#)
 - the setting of the core voltage [15](#)
 - refclk rate in MHz
 - measures [16](#)
 - regfile_bypass [67](#)
 - regfile_read [67](#)
 - regfile_read_2 [68](#)
 - regfile_read_3 [68](#)
 - regfile_write [68](#)
 - regfile01 [67](#)
 - regfile2 [67](#)
 - regfile3 [67](#)
- registering
 - a board class [39](#)
 - board class using auto-discovery [39](#)
- regression
 - scripts [31](#)
 - tests [32](#)
- regs [17, 19, 48](#)
- related documents [viii](#)
- returning
 - the core's current clock rate [15](#)
 - the DDR3 voltage [15](#)
 - the MAC number associated with this channel [23](#)
 - the setting of the core voltage [15](#)
 - the status of one link [24](#)
- revision ID of TILE-Gx processor
 - identifying [16](#)
- rsh_i2cm[n] [17](#)
- rsh_i2cs [17](#)
- rsh_uart[n] [17](#)
- rshim [17](#)
- Rshim I2C master commands [25](#)
- RSHIM_COORD SPR [ix](#)
- rshim.read(channel, address) [48](#)
- rshim.srom_read(address, num_words) [49](#)
- rshim.srom_write(address, num_words) [49](#)
- rshim.test() [47](#)
- rshim.write(channel, address, data) [48](#)
- run_test("testname") [47](#)
- run_tests() [34](#)
- running
 - basic aliveness test [15](#)
 - BTK tests [31](#)
 - qualified tests [73](#)
- S**
 - sample tests [31](#)
 - selecting
 - board type
 - using --board [12](#)
 - log directory with --log [13](#)
 - test phases [74](#)
 - TILE-Gx serial UART device with --serial-device [11](#)
 - self tests [33](#)
 - self-test [33](#)
 - SerDes/PCS loopback
 - enabling [24](#)
 - serial [9, 11, 53](#)
 - serial boot [51](#)
 - serial flag [6](#)
 - serial number [33](#)
 - Serial Presence Detect
 - See SPD
 - serial-bmc [11, 11](#)
 - serial-device [6, 9, 11](#)
 - set_context [34](#)
 - setting
 - core clock rates and corresponding voltages [15](#)

- setting (*continued*)
 - GPIO (high) [29](#)
 - logging and print levels [33](#)
 - the chip's core clock rate [15](#)
 - the state of GPIO pins [29](#)
- shared_mem [71](#)
- shim_AMT_test [68](#)
- show_test [16](#)
- simple_dll_test [68](#)
- single link
 - returning status [24](#)
- single port
 - disabling [22](#)
- single-UART booting [51](#)
- soft reset
 - issuing to TILE-Gx [15](#)
- software
 - installation [3](#)
 - drivers [3](#)
- SPD [43](#)
- spd_checksum [60](#)
- Special Purpose Registers
 - See SPRs
- specifying
 - the core voltage [15](#)
 - the DDR3 voltage [15](#)
 - the serial UART device be used to interact with the BMC console [11](#)
- speed [54](#)
- SPI ROM
 - writing the BIB to the SPI ROM [39](#)
- SPRs [18, 63](#)
- srom [61](#)
- srom_destructive [61](#)
- SROMs
 - adding new [40](#)
- StreamIO operations
 - configuring a port [22](#)
- striping mode [48](#)
- subclass
 - adding new subclasses to the new board [38](#)
- summary of state of all the MACs
 - printing [21](#)
- support [ix](#)
- suppress
 - output displays [35](#)
 - prints [35](#)
- sys [13](#)
- system requirements [3](#)
- T**
- technical or customer support [ix](#)
- temp [29](#)
- temp.read_all_remote() [28](#)
- temperature sensors
 - See temp
- TempSensor class [41](#)
- test
 - categories
 - basic [31](#)
 - Phase1 [31](#)
 - Phase2 [31](#)
 - dictionary [31](#)
 - Logs [34](#)
 - phases
 - selecting [74](#)
 - qualifiers [73](#)
 - BIB [75](#)
 - BMC [75](#)
 - board [74](#)
 - characterization [74](#)
 - chip [74](#)
 - custom [75](#)
 - defined [73](#)
 - destructive [75](#)
 - development [74](#)
 - enet [74](#)
 - example [74](#)
 - ext [75](#)
 - gbe [74](#)
 - gpio [74](#)
 - internal [74](#)
 - IPMI [75](#)
 - linux [75](#)
 - loopback [74](#)
 - mac [74](#)
 - manual [75](#)
 - margin [75](#)
 - memory [74](#)
 - mpipe [74](#)
 - multi [75](#)
 - Network [75](#)
 - PCIE [75](#)
 - phy [74](#)
 - production [75](#)
 - rom [74](#)
 - serdes [74](#)
 - table [74](#)
 - UART [75](#)
 - xaui [74](#)
- test() [45](#)
- testing
 - components [47](#)
- tests
 - BTK test list [59](#)
 - Linux [31](#)
 - listing all available tests [16](#)
 - regression [32](#)
 - reserved for TILE-Gx72 only [72](#)
 - running qualified tests [73](#)
 - self [33](#)
- Tile
 - general purpose registers [19](#)
 - Special Purpose Registers [18](#)

- tile
 - quiescing [18](#)
 - state
 - dumping [18](#)
 - unquiescing [18](#)
- tile(x,y) [17, 20](#)
- tile(x,y).dtlb.dump() [20](#)
- tile(x,y).dump(verbose=True) [18](#)
- tile(x,y).home_map.dump() [20](#)
- tile(x,y).itlb.dump() [20](#)
- tile(x,y).l1d.dump() [20](#)
- tile(x,y).pa_trace() [18](#)
- tile(x,y).quiesce() [18](#)
- tile(x,y).read_va(va) [18](#)
- tile(x,y).resume() [18](#)
- tile-btk [9](#)
 - described [10](#)
 - dev empower3
 - described [10](#)
 - dev none
 - described [10](#)
 - dev pci0
 - described [10](#)
- tile-dev [52](#)
- TILE-Gx node access [11](#)
- TILE-Gx72 only tests [72](#)
- tile-mkboot [51](#)
- TILExtreme-Gx node access [11](#)
- tlr.dump_msh_regs(shim_x, shim_y) [48](#)
- tlr.mem_init() [47](#)
- tlr.msh_diag_rd(interface_num, address) [48](#)
- tlr.msh_diag_wr(interface_num, address, data) [48](#)
- tlr.test_init() [47](#)
- tlr.tile_init() [47](#)
- top-level BTK objects
 - comp0 [17](#)
 - crypto0 [17](#)
 - gpio BTK object [17](#)
 - rsh_12cm[n] [17](#)
 - rsh_i2cs [17](#)
 - rsh_uart[n] [17](#)
 - rshim [17](#)
- top-level BTK objects (*continued*)
 - tile(x,y) [17](#)
 - trio[n] [17](#)
- training the associated PCIe port [21](#)
- TRIO
 - commands [21](#)
 - MAC commands [21](#)
 - RC commands [22](#)
- TRIO RC commands
 - m [22](#)
- trio_loopback [71](#)
- trio[n] [17](#)
- U**
- UART
 - configuration interface pins [3](#)
 - protocol mode [58](#)
- udn_conn [68](#)
- uncacheable [72](#)
- underscore character [27](#)
- unquiescing the tile [18](#)
- user interface
 - bringing up [9](#)
- using
 - BTK after boot [57](#)
 - customized Python code [14](#)
- V**
- vendor ID
 - finding matching devices(s) [23](#)
- verifying GbE packet counters [24](#)
- W**
- WIP bit [ix](#)
- worker tiles [72](#)
- write function [25](#)
- writing
 - MDIO PHY register using clause 22 [24](#)
 - MDIO PHY register using clause 45 [24](#)
- writing the BIB to a SPI ROM [39](#)
- X**
- xdn_demux [68](#)

